
Internship Thesis — Master 2 in Bioinformatics

A Standardized Pipeline for the Assembly and Annotation of Multiple Algal Genomes

Maiwen DEMEULLE

March - August 2025

Internship Supervisor: Louis GRAF

Academic Advisor: Alain DENISE

Educational Institution: Université Paris-Saclay / AMI2B M2 Bio-informatique

Host Organization: Laboratory of Computational Quantitative and Synthetic Biology (CQSB)



Contents

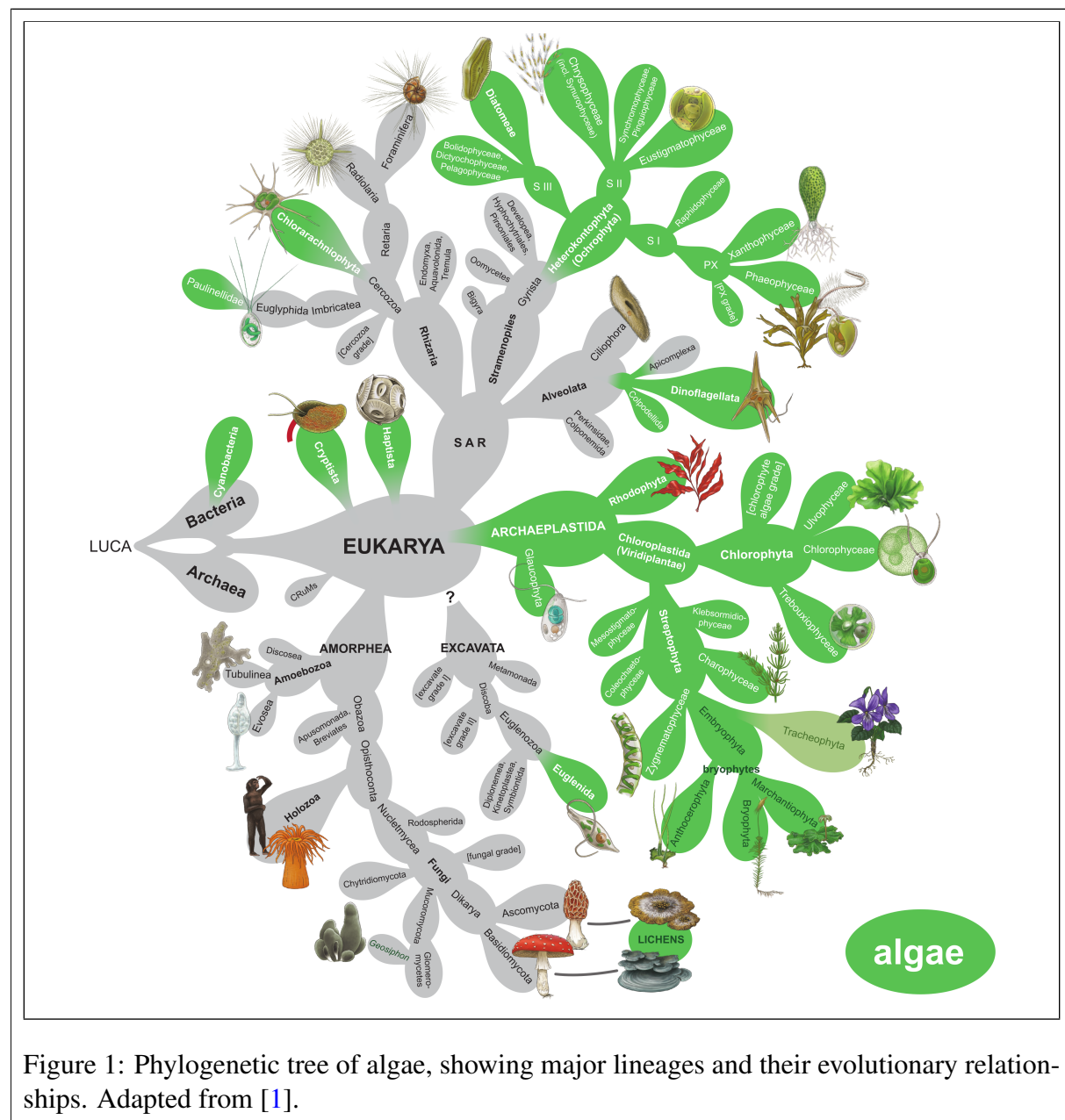
1	Introduction	1
1.1	Algae: A Diverse Group of Photosynthetic Eukaryotes	1
1.2	HEAL: Horizontal Evolution of Algal Lifestyles	3
1.3	Sequencing technologies and genome assembly	3
1.4	Problematic and Objectives	5
2	Methods	6
2.1	Computing Resources	6
2.2	Biological Dataset	6
2.3	Genome Assembly	6
2.3.1	Basecalling	6
2.3.2	Read Correction	7
2.4	Reads decontamination	7
2.4.1	V1	7
2.4.2	V2	7
2.4.3	V3	7
2.4.4	Assembler Comparison	7
2.4.5	Post-Assembly Correction	8
2.4.6	Assembly Quality Assessment	8
2.5	Annotation	8
2.5.1	Transcriptome Assembly	8
2.5.2	RNAseq Mapping	9
2.5.3	Repeat Identification and Masking	9
2.5.4	Structural Annotation	9
2.5.5	Functional Annotation	9
2.5.6	Quality Assessment	9
2.6	Analysis	9
3	Results and Discussion	10
3.1	Benchmarking on RCC7840	10
3.1.1	Correcting reads	10
3.1.2	Read decontamination	12
3.1.3	Genome Assembly	13
3.1.4	Assembly Polishing	14
3.1.5	Genome Annotation	15
3.2	Assembly and Annotation Pipeline	16
3.3	Application of the Assembly and Annotation pipelines to a broad range of algal species	18
3.4	Synten Analysis	20
4	Conclusion	24
5	Internship Assessment and Personal Contributions	25
	References	26

A	Supplementary Figures	31
B	Supplementary Tables	33
C	Commands	35
C.1	Assembly	35
C.1.1	Dorado	35
C.1.2	Herro	35
C.1.3	Dexchat	35
C.1.4	Binning	35
	C.1.4.1 Binning v1	35
	C.1.4.2 Binning v2	36
	C.1.4.3 Binning v3	36
C.1.5	Flye	38
C.1.6	Canu	38
C.1.7	Masurca	38
C.1.8	NextDenovo	39
C.1.9	hifiasm	39
C.1.10	Raven	39
C.1.11	BUSCO	39
C.1.12	Correction Assembly	39
C.2	Annotation	40
C.2.1	Fastp and rnaspades	40
C.2.2	STAR	40
C.2.3	Renaming and sorting	40
C.2.4	Repeat	41
C.2.5	Braker3	41
C.2.6	Funannotate (Structural Annotation)	41
C.2.7	Functional Annotation	41
	C.2.7.1 Egglog-mapper	41
	C.2.7.2 InterProScan	42
	C.2.7.3 SignalP6	42
	C.2.7.4 Phobius	42
	C.2.7.5 Funnannotate	42
C.2.8	rRNA prediction	42
C.2.9	Annotation statistics	42

1 Introduction

1.1 Algae: A Diverse Group of Photosynthetic Eukaryotes

The term “algae” regroups a wide range of unicellular or multicellular organisms that absorb and lose water passively (hydro-passive) and perform **oxygenic photosynthesis** [1]. They are found across multiple supergroups within the eukaryotic tree of life (Figure 1). Their diversity spans a wide range of morphologies, ecological niches, and evolutionary lineages and therefore the algae are a highly paraphyletic group. Photosynthesis spread across the tree of life not through vertical inheritance but through endosymbiotic events (Figure 1) [1]. Beyond their taxonomic and morphologic diversities, algae play fundamental ecological roles. They are major primary producers in aquatic ecosystems, **contributing half of the atmospheric oxygen through photosynthesis and forming the base of food webs** [2, 3].



Although algae spans almost all lineages of eukaryotes, genomic data is disproportionately scarce for many lineages with most of the data concentrated in a handful of lineages like the green or red algae (Figure 2). These differences can not be explained by the abundance of a certain algal group (e.g. the most abundant group of the diatoms is only represented by 10 genomes) or ecological importance. Indeed, despite their ecological and evolutionary importance, algal genomes remain underrepresented in sequencing databases (Figure 2). Indeed, many species are difficult or even impossible to cultivate under laboratory conditions [4], limiting access to their DNA through traditional culture-based methods. Even when DNA can be extracted, generating good quality reads is itself challenging due to biological factors (multi-layered cell walls, secondary metabolites such as polysaccharides). This **inability to cultivate** them makes genome sequencing difficult and leaves a **large fraction of algal biodiversity unexplored by genomic approaches**. However, increasing accessibility to genomic data and improvements in sequencing technologies are helping to close this gap.

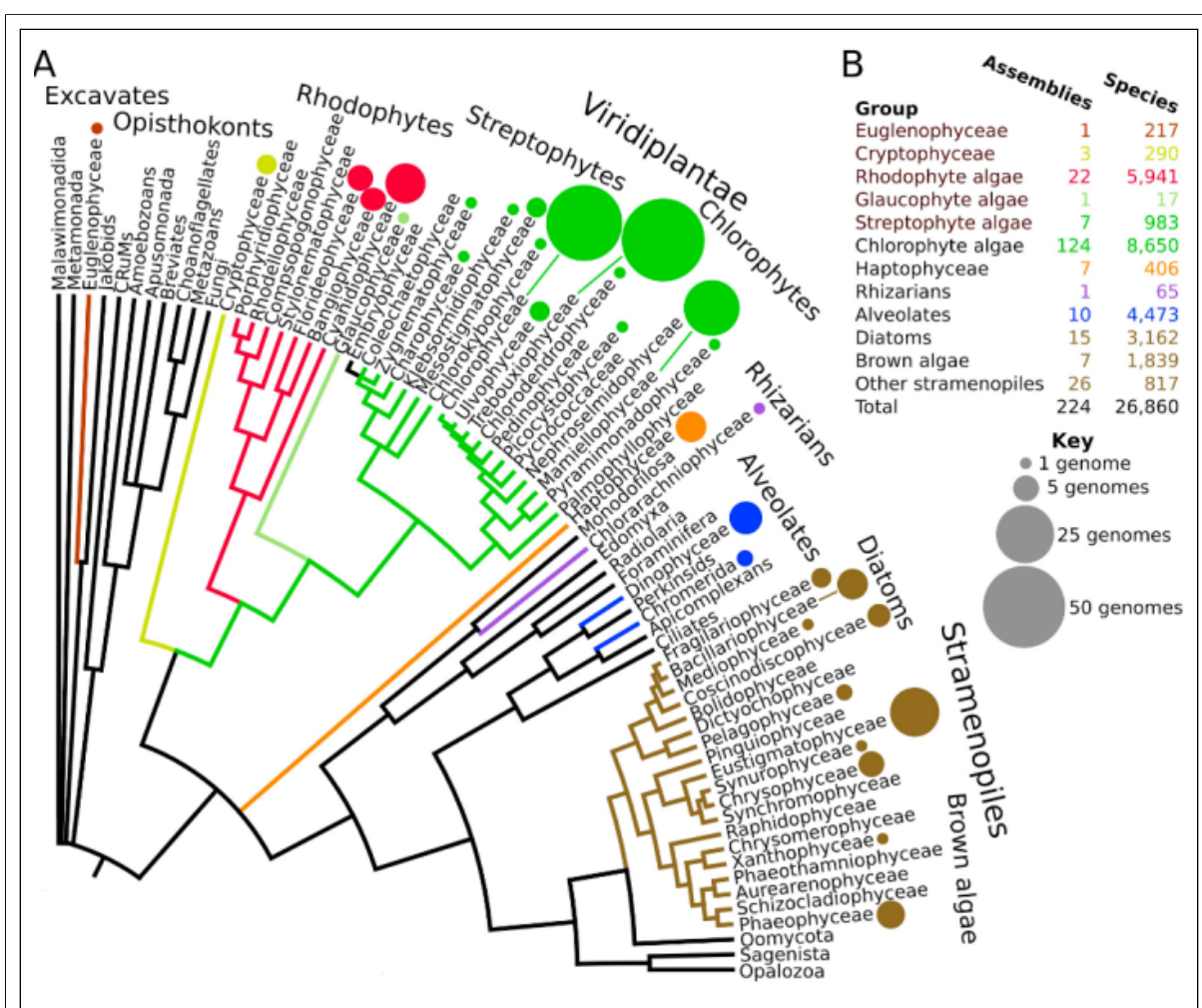


Figure 2: Phylogenetic distribution of algal genome assemblies.

(A) A simplified eukaryotic tree showing taxonomic groups that include algae, colored by major algal lineages. Circle size represents the number of sequenced genomes per group. (B) Summary table of genome assembly counts and known species per group. Algal groups such as Chlorophytes show significant sequencing progress, while others remain underrepresented. Adapted from [5].

While transcriptomic approaches such as RNA-seq can provide valuable insights into gene expression, they cannot fully substitute for genome sequencing as it can only capture expressed

genes in a context dependent manner. Therefore, **comprehensive genomic** data remains essential to access the whole gene library, reveal intron-exon structures, and study synteny and transposable elements. Moreover, genome data provide critical evidence to identify horizontal gene transfer (HGT) events and distinguish them from contamination, something that transcriptomic data alone cannot reliably achieve [6].

1.2 HEAL: Horizontal Evolution of Algal Lifestyles

The HEAL [7] team is part of the CQSB (Computational Quantitative and Synthetic Biology) research unit at the IBPS (Institut de Biologie Paris-Seine) at Sorbonne University. The team studies the evolution of algal lifestyles, focusing on how gene transfer and endosymbiosis events have shaped their diversities, ecological adaptations and resilience.

Combining experimental approaches such as proteomics, mutagenesis and cellular localisation with computational methods including genomics, phylogenomics and metagenomics, the team explores the evolution of chloroplast's proteomes, the role of novel chloroplast-targeted proteins, and the ecological and evolutionary impact of horizontal interactions across marine and polar environments. These projects offer valuable findings into the biology and ecology of algae, **which are major contributors to global photosynthesis and oceanic ecosystems**.

In order to achieve this objective, the team is engaged in the study of a wide range of algal species particularly through their Arctic project. This initiative explores the **genomic diversity** of a set of algae from the Arctic region and aims to understand the evolutionary history of these species. **This project relies on the sequencing, assembly and annotation of high quality genomes**.

1.3 Sequencing technologies and genome assembly

First generation sequencing, primarily represented by **Sanger sequencing**, was the foundation technologies in DNA sequencing. In 1987, fluorescence based Sanger sequencing machines could generate around 1000 bases per day [8]. In 1982, the GenBank database was launched, providing a repository for sequences. By the end of that year, more than half a million sequences had been archived in **GenBank**. Today, this number has grown exponentially: by 2025, the database contains over 5000 billion sequences, about 10 million times more [9]. This explosion of data is driven by advances in sequencing technologies, and highlights the pressing need for more powerful and efficient computational methods to analyze these sequences. The surge in sequence output has been fueled by major technological breakthroughs. In the mid-2000s, **second generation sequencing**, often referred to as **next-generation sequencing (NGS)**, introduced massively parallel sequencing of short reads, superseding Sanger sequencing as the dominant technology. Platforms such as Illumina, launched around 2006, provide read lengths typically ranging from 100 to 300 bases with high accuracy (error rate < 0.1%) [10]. Finally, third generation sequencing or single-molecule sequencing, such as PacBio and OxfordNanopore, allows for much longer reads, at best several hundred thousand bases at the cost of a higher error rate (5%) [11]. The advancement of sequencing technologies with reduced error rates and higher output volumes has diminished the impact of sequencing errors on analyses [12].

The democratization of long-read sequencing (e.g., PacificBiosciences Hifi, Oxford Nanopore Technologies) has revolutionized genome assembly, allowing for the assembly of larger and more complex genomes. Long reads can span entire repetitive regions, which are often problematic for short-read technologies [13]. However, they also come with higher error rates, necessitating additional correction steps.

Even once high-quality reads are obtained, there is still a long path from raw data to a genome ready to be analysed with biological questions. Transformation from raw sequencing data involves several complex and interdependent steps: basecalling, preprocessing, assembly, scaffolding, gap filling and polishing [14]. The preprocessing stage involves several steps, including the removal of low-quality reads and potential contaminants from non-target organisms (such as *Homo sapiens* and/or bacteria). Genome assembly is a computationally and methodologically complex task. Factors such as repetitive sequence [15], contamination and choice of assembly strategy play a critical role in assembly quality.

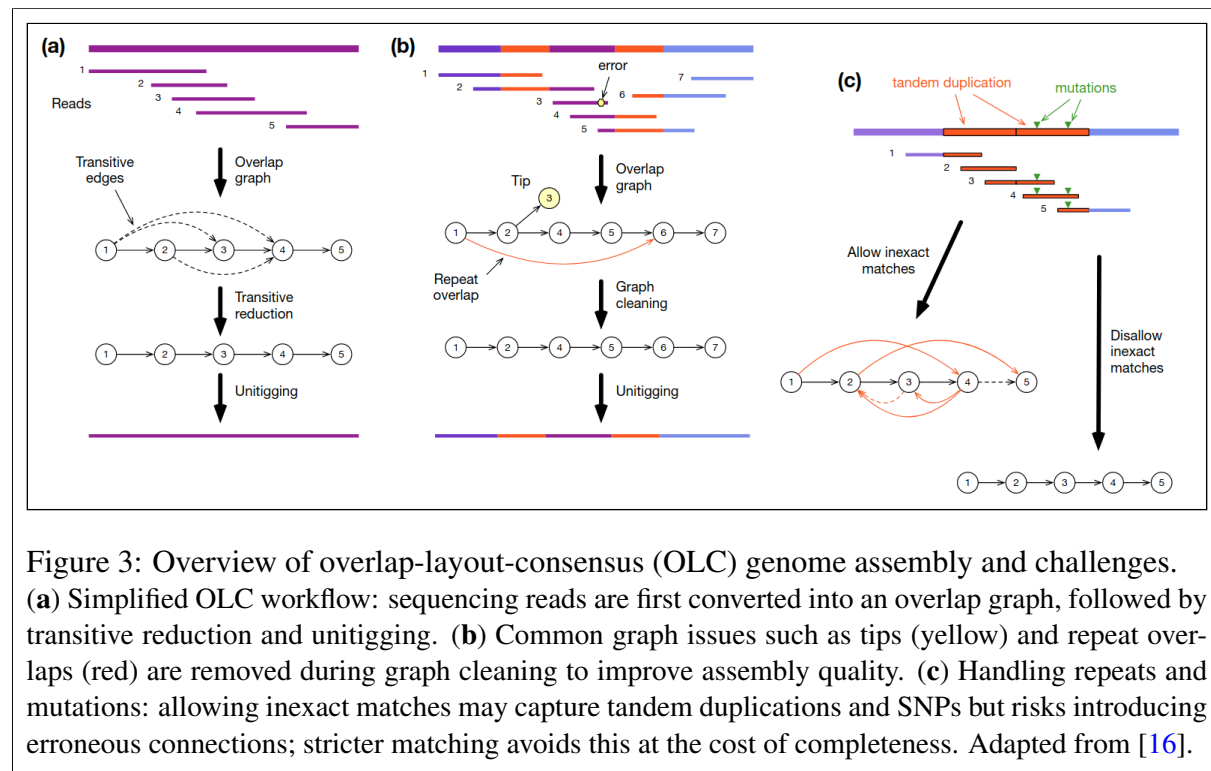


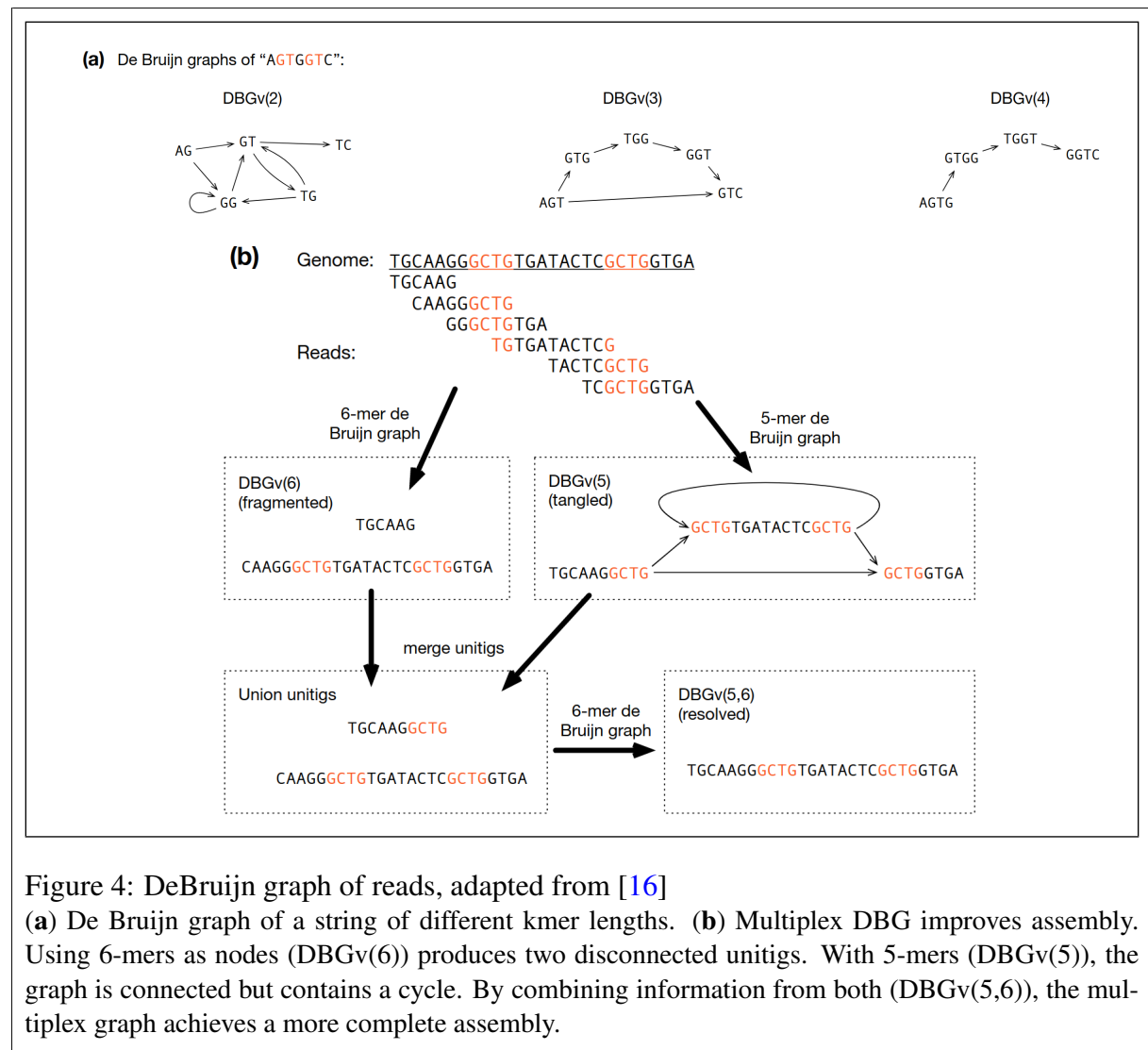
Figure 3: Overview of overlap-layout-consensus (OLC) genome assembly and challenges. (a) Simplified OLC workflow: sequencing reads are first converted into an overlap graph, followed by transitive reduction and unitigging. (b) Common graph issues such as tips (yellow) and repeat overlaps (red) are removed during graph cleaning to improve assembly quality. (c) Handling repeats and mutations: allowing inexact matches may capture tandem duplications and SNPs but risks introducing erroneous connections; stricter matching avoids this at the cost of completeness. Adapted from [16].

Most modern assemblers are graph-based, typically employing either a *de Bruijn graph* (DBG) or an *overlap-layout-consensus graph* (OLC) [16]. On one hand, in the OLC approach, each read is represented as a node, and edges connect **overlapping reads** (Figure 3). OLC approach is robust to sequencing errors and works well with variable read lengths but is computationally intensive due to the all-to-all overlap calculation.

On the other hand, the DBG method decomposes reads into smaller k-mers, constructing a graph where nodes represent unique k-mers and edges denote overlaps of length $k-1$ (Fig. 4). This method is computationally efficient but more sensitive to sequencing errors.

The primary benefits of the DBG approach is its lower computational cost, since it avoids calculating overlaps as in the OLC methods.

Once a high-quality assembly is obtained, the next challenge is genome annotation. Annotating a genome involves identifying genes, regulatory elements, and other functional elements such as tRNA and rRNA genes. This process typically involves first identifying non coding elements such as repeats. Once repeats are masked, gene prediction tools can be used to identify protein coding genes. Then biological information and function can be attached to these elements [14]. Each annotation step presents challenges, especially in poorly studied or highly divergent algal genomes, from distinguishing true genes from sequencing artifacts to assigning accurate functional information.



1.4 Problematic and Objectives

Despite major advances in sequencing technologies and new methods, high quality and annotated algal genomes remain scarce. This scarcity is not due to a lack of environmental and evolutionary importance but rather to technical challenges such as cultivation difficulties, complex genome structure and frequent contamination in environmental samples. These factors make assembling and annotating algal genomes particularly challenging. Without more complete and accurate assemblies, our understanding of algal biodiversity remains limited.

To address these issues, my project proposes to design and implement a comprehensive bioinformatics pipeline for the assembly and annotation of algal genomes. This pipeline aim to:

- Process raw sequencing data from various platforms and handle possible contamination.
- Optimize genome assembly strategies specifically for algal genomes with their unique characteristics (e.g., variable genome sizes, repetitive elements, organellar genomes).
- Provide a flexible and modular framework that can be adapted to different algal species, genome complexities, and sequencing technologies.
- Provide accessible tools and documentation that can be applied by the broader research community.

2 Methods

Commands used to run each software are described in the appendix.

2.1 Computing Resources

We primarily used two different computing resources, depending on availability on the cluster. The first was a local AlmaLinux 9.5 machine equipped with 128 GB of RAM, an Intel(R) Xeon(R) Silver 4214R CPU (2.40GHz, 48 cores) and a NVIDIA RTX A4000 GPU. The second resource was the Core Cluster from the Institut Français de Bioinformatique (IFB), composed of:

- 16 high-performance compute nodes (HPE Apollo 2000 XL225n Gen10+) with a total of 128 cores and 2 TB of RAM. Each node contains 2x AMD EPYC 7662 processors (3.1 GHz, 64 cores), with a total of 3 TB RAM available across these nodes.
- 68 standard compute nodes (DELL C6320), each with 28 cores and 256 GB of RAM, powered by 2x Intel Xeon E5-2695v3 CPUs (2.3 GHz, 14 cores).

Slurm, a job scheduling tool, dynamically allocated cores and memory depending on job requirements and resource availability at the time of execution.

2.2 Biological Dataset

Sequencing data was generated ahead of my internship by the HEAL Team at the CQSB. Briefly, algal cultures were maintained at 4°C in K/2 or F/2 media according to their respective preferences. Nucleic acids (i.e. DNA and RNA) were extracted from cell pellets from 50 to 250cL of culture in log phase. RNA was extracted using a TRIzol/Chloroform protocol with isopropanol precipitation. DNA was extracted following Chaux et al., 2024 [17] using a Phenol/Chloroform/Isoamyllic Acid protocol optimized for recovery of High Molecular Weight DNA (HMW DNA). In order to facilitate genome assemblies, DNA was sequenced using both short and long reads technologies for each algal species. RNA was sequenced with short reads technology only. Long reads were sequenced in house using the Oxford Nanopore Technologies (ONT) on either a MinION or a PromethION platform using R10.4.1 flow cells with libraries prepared either with the SQK-LSK114 ligation kit (i.e. for runs with a single species) or the SQK-NBD111-24 native barcoding kit (i.e. for runs with multiple species). Short reads sequencing was outsourced to the Beijing Genome Institute (BGI) using the DNBseq platform.

2.3 Genome Assembly

To determine the optimal tools and parameters for our algal assembly pipeline, I conducted a benchmarking of multiple assemblers and read correction methods on one species. I opted to use the species with the smallest predicted genome size. Prior to my internship, genome sizes and repeat content for each algal species were predicted using DNA short reads with the kmer frequency distribution model implemented in **GenomeScope2.0** [18].

2.3.1 Basecalling

Nanopore sequencing data is provided as raw electrical signals (pod5 files). These signals need to be converted into nucleotide sequences through a process called basecalling. This process

involves using machine learning models to predict the most likely sequence of nucleotides from the raw signal data. **Dorado** [19] is the provided basecalling software for ONT reads. I used the 0.7.3+6e6c45c version of dorado with the most accurate model, sup, but slower than the fast and hac models.

2.3.2 Read Correction

We compared two different read correction approaches:

- **Herro** [20]: Implemented in the dorado software.
- **Dechat** [21]: A specialized tool for repeat-aware error correction.

2.4 Reads decontamination

The strategy for reads decontamination was to reduce the presence of non-target sequences to improve the accuracy of genome reconstruction. I combined taxonomic decontamination, to identify and remove contaminant reads, with targeted binning to separate mitochondrial and plastid sequences from the dataset. I tested three successive strategies to progressively refine the pipeline. We will call them v1, v2 and v3 along this report and v1 reads will refer to the reads after the v1 approach, v2 reads to the reads after the v2 approach and so on. A schematic summarizing these decontamination and binning steps is provided in Supplementary Figure 1.

2.4.1 V1

Initial tests focused on long reads using the **metagenome workflow** v2.13.0 [22] developed by the EPI2ME platform. The kraken2 k-mer approach was selected for speed, using the PlusPFP-8 database. In this first strategy, only long reads flagged as "unclassified" were kept, while all other reads were removed. Short reads were not modified at this stage.

2.4.2 V2

The second strategy applied Kraken2 to long reads but kept all reads except those **flagged as bacteria**. Short reads remained untouched. This approach allowed retention of more long reads while still removing potential contaminants.

2.4.3 V3

In the final version, **Kraken2** was also applied to short reads outside the EPI2ME workflow, keeping only unclassified short reads. A custom database combining the standard Kraken2 database with fungal sequences from PlusPFP was used. Organellar genomes were then assembled from short reads using **NovoPlasty** (v4.3.5) [23]. Reads mapping to the assembled mitochondrial and plastid genomes (both long and short) were removed, producing a clean nuclear genome dataset for assembly.

2.4.4 Assembler Comparison

I evaluated several long-read assemblers specifically for their performance on our algal genomes:

Flye 2.9.5-b1801 [24] assembles long reads using a repeat graph strategy, which offers improved handling of repetitive regions compared to classical de Bruijn graphs.

Canu 2.3 [25], developed as a fork of the Celera Assembler, targets noisy long reads with a correction-first strategy, followed by assembly using overlap-based methods.

MaSuRCA 4.1 [26] integrates short and long reads in a hybrid approach that combines *Overlap-Layout-Consensus* and *de Bruijn Graph* principles. Short reads are first converted into *super-reads*, which are longer and more accurate, and subsequently used to correct long reads. Masurca was run with two different options: either using Flye as final assembler or CABOG (see appendix for commands).

NextDenovo 2.5.2 [27] follows a correct-then-assemble strategy based on de Bruijn graphs, similar in spirit to Canu, but optimized for modern long-read data.

hifiasm 0.24.0-r703 [28] is a tool designed for assembling HiFi reads, but it can also be used with ONT reads. It uses a graph-based approach to resolve haplotypes and produce a high-quality assembly.

raven 1.6.1 [29] is a long-read assembler following a layout-graph-consensus approach.

2.4.5 Post-Assembly Correction

We tried the correction pipeline described in [30] to correct the assembly using short and long reads. This protocol uses one round of Racon 1.4.20 [31] long read correction, followed by 4 rounds of Medaka 2.0.1 [32] long read correction and 4 rounds of NextPolish 1.4.1[33], long and short reads correction. The three tools were run using default parameters.

2.4.6 Assembly Quality Assessment

For each assembler combination, we evaluated assembly quality using:

- **BUSCO** 5.8.3 [34] with the eukaryota lineage dataset (odb12) to assess gene completeness. BUSCO evaluate completeness by searching gene in the assembly from single copy orthologous gene dataset and reporting them as complete, fragmented or missing.
- **L50/N50 metric**: Length of the shortest contig covering 50% of the genome. Higher N50 indicates a more contiguous assembly.
- **Number of contigs**: Ideally, an assembly with fewer contigs would be less fragmented than one with more contigs.
- **Number of contigs larger than 1Mb**
- **Total assembly size**: Comparison with expected genome sizes.

2.5 Annotation

2.5.1 Transcriptome Assembly

Prior to the annotation, we had to assemble the RNAseq reads into transcripts. We first used **fastp** 0.24.0 [35] to preprocess the fastq files, removing low-quality reads and trimming adapters. Then we used rnaspades from the **SPAdes** 3.15.5 assembler [36] with default settings to assemble the transcripts from the processed reads.

2.5.2 RNAseq Mapping

When RNAseq data is available, we can use it to improve the annotation quality. We then used **STAR** 2.7.11b [37] to map the RNAseq reads to the assembled genome and generate a BAM file.

2.5.3 Repeat Identification and Masking

RepeatModeler 2.0.5 [38] is a tool for de novo repeats identification and classification. It builds a library of repeats specific to a genome assembly. We then used **RepeatMasker** 4.1.8 [39] to mask the repeats of the RepeatModeler library in the assembly.

2.5.4 Structural Annotation

Structural gene prediction used a combination of evidence-based and ab initio approaches. **BRAKER3** 3.0.8 [40] was run with RNA-seq alignments when available and a default protein database, internally using **GeneMark-ETP** [41] and **AUGUSTUS** [42] to predict gene models. Additional ab initio predictions were generated using **GlimmerHMM** [43] and **SNAP** [44] and transcript-based predictions were obtained using **PASA** [45] (all implemented within the **Funannotate pipeline** [46]). **Funannotate** 1.8.17 was used as a framework, integrating prediction outputs from BRAKER3, PASA and the ab initio predictors, and merging them into a final, non-redundant gene set using **EvidenceModeler**.

2.5.5 Functional Annotation

Functional characterization of predicted proteins employed several databases and resources. **InterProScan** 5.67-99.0 [47] identified protein domains, motifs and families via databases such as Pfam, SMART and PROSITE. **EggNOG-mapper** 2.1.12 [48] assigned functions based on orthology relationships from the eggNOG database (5.0.2 release). **Phobius** 1.01 [49] predicted transmembrane topology and signal peptides, while **SignalP6** 6.0h [50] identified signal peptides and their cleavage sites. **Barrnap** 0.9 [51] annotated ribosomal RNA (rRNA) genes by comparison to a database of known rRNA sequences. **Funannotate** was then used as an integration layer, parsing results from InterProScan, EggNOG-mapper, Phobius and SignalP6 to generate product name and functional classifications (e.g. GO).

2.5.6 Quality Assessment

Statistics on the annotation were generated using the Toolkit **AGAT** 1.4.1 [52].

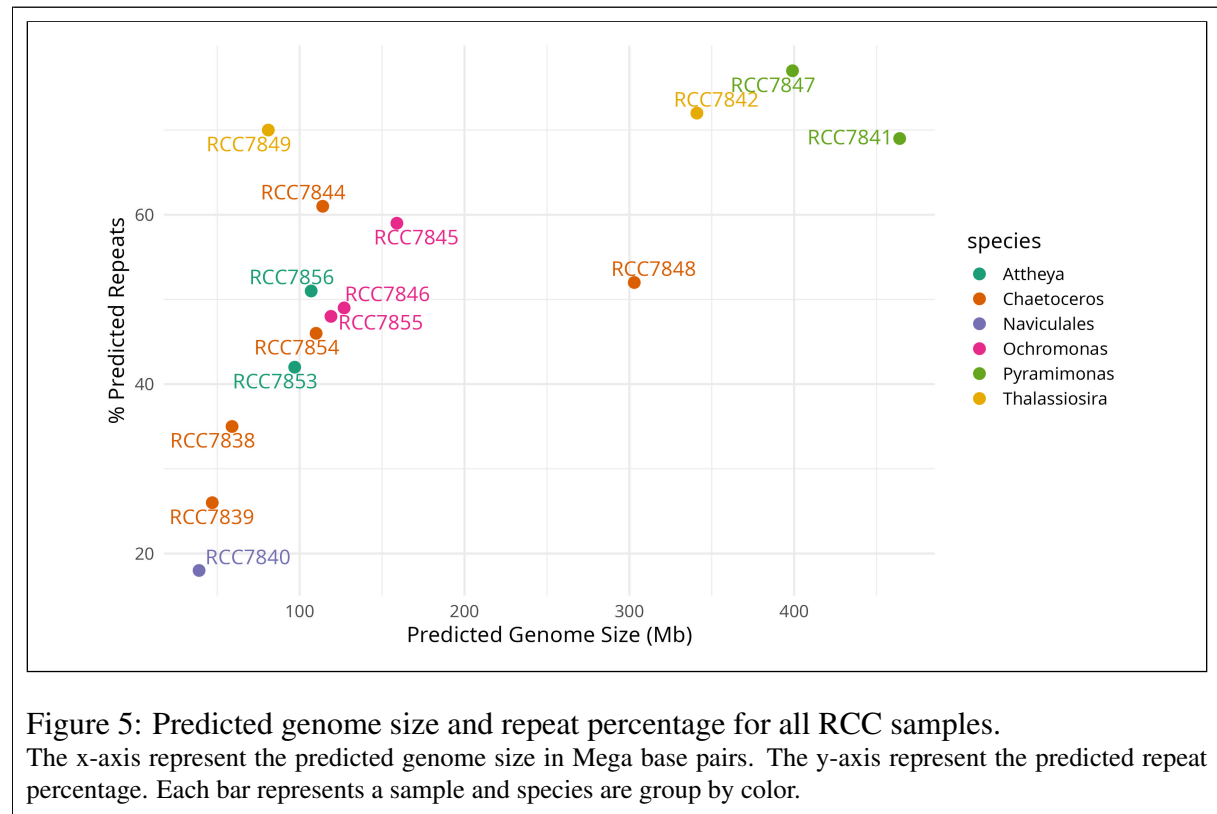
2.6 Analysis

Synteny blocks between genomes were identified in contigs longer than 1Mb for the algal strains RCC7838, RCC7839, RCC7844, RCC7848 and the publicly available genome of *Chaetoceros tenuissimus* [GCA_021927905.1](#). Homology between the genes models encoded in all the contigs longer than 1Mb was searched using MMseqs2 version 7f5c0003a913 [53] and analysed using **MCScanX** 1.0.0 [54] for synteny blocks detection. Synteny graph was visualized using the Synteny Browser **SynVisio** [55]. We also use the **Mauve** 2.4.0 [56] software to perform whole-genome alignments on mitochondrial and plastid genomes.

3 Results and Discussion

3.1 Benchmarking on RCC7840

I choose one sample to perform benchmarking to determine which tools perform best for assembling and annotating algal genomes. I selected RCC7840 as it had the lowest predicted genome size and repeats percentage (Figure 5, Supplementary Table 1) so it would be easier to assemble and annotate.



3.1.1 Correcting reads

A total of 653,137 long reads, representing 4.8 Gigabase (Gb), were sequenced for the RCC7840 sample. Reads showed a N50 length of 12,848 base pairs (bp). In order to enhance the quality of raw reads before assembly, we first explored long read correction tools.

-	Raw	Herro	Dechat
mean length	7 393.3	11 249.8	7 410.4
median length	5 021	9 589	5 032
reads	653 137	328 171	652 907
N50	12 848	13 410	12 876
total size	4 828 846 532	3 691 848 143	4 838 298 676

Table 1: Statistics of the reads of RCC7840 after correction.

We tested two correction tools: Herro and Dechat. Herro significantly reduced the number of reads (by nearly half compared to Dechat), but yielded higher N50 and mean read lengths (Table 1). Dechat preserved most of the original reads and produced statistics similar to the raw dataset. Figure 6 illustrates the differences in read length distributions. Herro filtered out reads shorter than approximately 10,000 bases, resulting in a distribution skewed toward longer reads. In contrast, Dechat maintained a broader range of read lengths. This length based filtering by Herro explains its higher N50 value.



Figure 6: Length distribution of the reads of RCC7840 after correction.

Plot generated with *Nanoplot*. The x-axis represent the read length in log scale and the y-axis represent the number of reads. On the left, Dechat corrected reads and on the right, Herro corrected reads.

To evaluate the impact of these corrections, I assembled the corrected reads using Flye. Compared to the assembly done with uncorrected reads, both tools seem to have worsened the assembly. Indeed, both assemblies are more fragmented and have lower N50 values. The Herro-corrected dataset produced a less fragmented assembly with a higher N50 compared to Dechat (Table 2). The total assembly size and largest contig length were similar for both corrections, suggesting Herro's strategy improved contiguity without sacrificing completeness.

Correction	Assembler	fragments	N50	Size	largest fragments
herro	flye	309	785 642	47 228 517	4 245 254
dechat	flye	365	564 778	47 808 010	4 251 011
None	flye	294	1 008 793	46 646 859	4 246 806

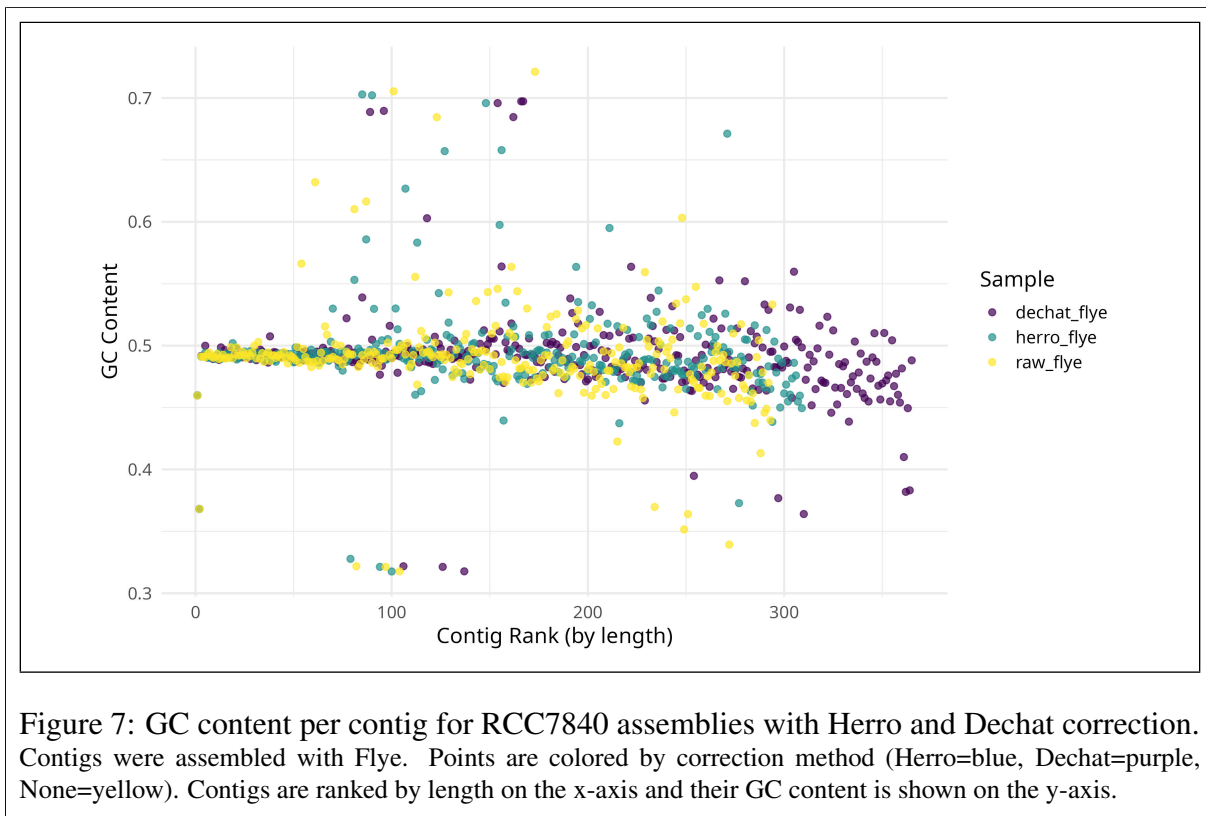
Table 2: Statistics of the assemblies of RCC7840 using Flye with different correction tools.

Although Herro correction produced slightly higher N50 values than Dechat in our dataset, both pre-correction methods yielded assemblies that were more fragmented than those generated from uncorrected reads. These results are consistent with recent evaluations of ONT R10.4.1 data which demonstrate that raw reads can achieve error rates below 1% and deliver assemblies without requiring pre-assembly correction [57]. In our dataset, pre-correction did not consistently improve contiguity or completeness and increased computational demands. Consequently, Herro correction was retained only as an optional step in the pipeline.

3.1.2 Read decontamination

Decontamination is crucial in eukaryotic genome assemblies because the presence of bacterial or other non-target sequences can lead to chimeric contigs and misassemblies [58].

To investigate contamination, GC content of the assembled contigs was analysed (Figure 7). Although the overall GC distribution was similar between assemblies, some contigs exhibited outlier GC values, suggesting possible contamination. To address this, I applied the decontamination and binning strategies described in the Methods section.



NOVOPlasty was used to assemble the mitochondrial and chloroplast genomes from the V3 binning. The mitochondrial genome assembled into a single circular contig of 36,933 bp. The chloroplast genome assembled into three contigs of 81,747 bp, 43,713 bp, and 43,849 bp.

On the 80,079,365 paired reads, 3.80% aligned to the assembled mitochondrial and chloroplast genomes. We retained only reads that were paired and did not align (samtools flag -f 5) representing 96.17% of the total reads. From these retained reads, decontamination with Kraken removed 2.80%.

NOVOPlasty was used to assemble the mitochondrial and chloroplast genomes from the V3 binning. The mitochondrial genome assembled into a single circular contig of 36,933 bp. The chloroplast genome assembled into three contigs of 81,747 bp, 43,713 bp, and 43,849 bp. These three plastid contigs were probably the results of the presence of the inverted repeat (IR) sequence [59, 60] that make the chloroplast genome harder to resolve.

On the 80,079,365 paired reads, 3.80% aligned to the assembled mitochondrial and chloroplast genomes. We retained only reads that were paired and did not align (samtools flag -f 5) representing 96.17% of the total reads. From these retained reads, decontamination with Kraken removed 2.80%.

As shown in Table 3, binning significantly reduced the number and total size of reads. The v1 version of binning was the less conservative, removing 150,000 reads. Assemblies were

-	Raw	Binning v1	Binning v2	Binning v3
mean length	7 393.3	6 225.4	6 297	7 329.3
median length	5 021	4 239	4 287	5 024
reads	653 137	502 788	518 950	568 273
N50	12 848	10 872	11 022	12 746
total size	4 828 846 532	3 130 036 670	3 267 825 247	4 165 023 485
fragments	91	108	82	67
N50	709 649	670,437	927 216	972 742
Size	40.1	38,4	39.8	38.7
largest fragments	2 819 932	2 272 690	3 617 955	3 458 164

Table 3: Statistics of reads and assemblies of RCC7840 with binning.
Assemblies were done with MaSuRCA assembler (cabog version).

generated with MaSuRCA (cabog) to evaluate the impact of reads decontamination and binning on assembly quality, particularly binning on short reads as MaSuRCA is a hybrid assembler. In version 1, the read N50 decreased by approximately 2 kb, whereas in version 3, the read N50 was closer to the raw values. The important difference between binning v1 and v2 is visible in their respective assembly. Although the reads N50 of these two is slightly better with v2, the assembly based on V2 reads is much less fragmented (82 contigs against 108 for the assembly based on v1 reads). The assembly N50 also increased from 670 kb to 927 kb. Total assembly size remained relatively stable across binning suggesting that no genomic content was lost to decontamination.

Based on these results, Binning v3 was selected as the strategy in the pipeline.

3.1.3 Genome Assembly

I tested several assemblers, including Canu, raven, MaSuRCA, NextDenovo and hifiasm. All provided an assembly except NextDenovo which failed to run. Choosing the assembler was done alongside the testing of binning strategies hence not all assemblers were tested with all binning strategies. Assemblers were run on error-corrected reads when it was possible for the assembler to accept a FASTA file; otherwise, they were run with raw FASTQ reads.

Assembly statistics for RCC7840 using various assemblers and correction strategies are summarised in Table 4. The hybrid assembler MaSuRCA can be run with either the Flye or cabog final steps assembly. While the author of MaSuRCA recommends using the Flye assembler for hybrid assembly, and the cabog assembler if only short reads are available, we did not observe superior performance with Flye. In our case, the cabog assemblies were better, producing assemblies with fewer fragments and higher contiguity. The best assembly was obtained with MaSuRCA cabog + binning v3, yielding 67 fragments, an N50 of 972,742 bp, and the second longest contig of 3.4 Mb. By contrast, Raven (Herro-corrected reads) produced the most fragmented assembly (607 contigs, lowest N50). Assembly sizes also varied, with Canu (binning v2) producing the largest genome size (53.0 Mb). BUSCO analysis indicated that all assemblies had broadly similar completeness (>90%) consistent with successful recovery of most expected conserved genes. However, differences were observed in duplication rates: Canu assemblies displayed a noticeably higher proportion of duplicated BUSCO, which is in line with their larger assembly likely reflect redundant and uncollapsed haplotypes rather than true genome expansion. By contrast, the MaSuRCA cabog assemblies achieved high complete-

Processing	Assembler	fragments	N50	Size (Mb)	Max (Mb)
v1/herro	raven	607	146 149	39.6	1.145
v1/herro	flye	439	215 245	40.0	1.540
v1/none	flye	336	369 082	40.8	1.165
v1/None	hifiasm	308	353 870	42.4	1.632
v1/None	masurca (F)	258	528 246	39.2	1.770
v1/None	masurca (C)	108	670 437	38.4	2.272
v2/None	Canu	572	322 783	53.0	1.233
v2/None	masurca (C)	82	927 216	39.8	3.617
v3/None	masurca (C)	67	972 742	38.7	3.458

(a) Assembly statistics

Processing	Assembler	BUSCO Results
v1/None	flye	C:90.7% (S:86.8, D:3.9), F:5.4%, M:3.9%
v1/None	hifiasm	C:89.9% (S:85.3, D:4.7), F:5.4%, M:4.7%
v2/None	Canu	C:90.7% (S:66.7, D:24.0), F:5.4%, M:3.9%

(b) BUSCO completeness results

Table 4: Statistics of the assemblies of RCC7840 with binning.
Processing format: binning version/reads correction method Reads passed to hifiasm and Masurca were not corrected as these software required raw fastq file. MaSuRCA variants: (F=Flye) and (C=cabog) refer to final assembly method.

ness (90.7% complete BUSCOs) with lower duplication and missing rates, supporting their superior quality.

In summary, MaSuRCA cabog with binning v3 provided the most contiguous and accurate assembly for RCC7840, balancing high N50 and long contigs with lower redundancy, making it the optimal choice for downstream annotation.

3.1.4 Assembly Polishing

The polishing was applied on an assembly generated by hifiasm based on the v1 reads. I combined 4 rounds of Racon, followed by 1 round of Medaka and 4 rounds of NextPolish.

Protocol	Size (Mb)	N50	Fragments	Max
Raw	42.403	353 870	308	1 632 807
Racon x 4	42.395	354 962	308	1 632 949
+ medaka x 1	42.384	354 836	308	1 631 166
+ NextPolish x 4	42.371	354 790	308	1 631 480

Table 5: Assembly statistics of RCC7840 with polishing.
The assembly was done with hifiasm assembler on v1 reads. Polishing was done with Racon, Medaka and NextPolish.

Assembly statistics remained essentially unchanged: the total length decreased only slightly (42.39 Mb → 42.37 Mb), the number of contigs was stable at 308, and the N50 varied only marginally (354,962 bp → 354,790 bp). Similarly, the size of the largest contig remained around 1.63 Mb (Table 5). These polishing steps required significant computational resources, with

little measurable benefit. Moreover, since no reference genome is available for all our samples, it is difficult to directly quantify the effect of polishing. Although metrics such as BUSCO could provide indirect insight, given the already good quality of the assembly (hybrid assembly of long and short reads), no major increase in BUSCO completeness is expected. For these reasons, polishing was retained in the pipeline only as an optional step, allowing its use for datasets where a clearer benefit might be achieved.

3.1.5 Genome Annotation

Using RepeatModeler, 325 repeat families were identified in the genome assembly. Using this library of repeats, RepeatMasker softmasked 13.49% of the genome of RCC7840. Results

Name	Number	Length (bp)	Percent (%)
Retroelements	1 109	522 377	1.31
LINEs	644	339 306	0.85
CRE/SLACS	16	1 534	<0.01
RTE/Bov-B	52	6 598	0.02
LTR elements	465	183 071	0.46
Ty1/Copia	394	156 122	0.39
Gypsy/DIRS1	26	8 413	0.02
DNA transposons	115	132 230	0.33
MULE-MuDR	35	27 856	0.07
Tourist/Harbinger	22	9 299	0.02
Unclassified	12 513	4 026 038	10.11
Total interspersed repeats		4 680 645	11.75
Small RNA	378	223 347	0.56
Simple repeats	10,603	447 031	1.12
Low complexity	500	23 155	0.06

Table 6: Repeat elements in the RCC7840 assembly.
Repeat element that were not found in the assembly have been omitted.

in Table 6, show the detail of classes of repeats found in the assembly. Only 1.31% of the assembly was classified as retroelements and 0.33% as DNA transposons, while a large fraction (10.11%) remained unclassified. This likely reflects the reliance of the annotation pipeline on repeat libraries from a few well known species, which may not capture highly diverged or novel repeat families. Simple repeats and small RNA elements accounted for 1.12% and 0.56% of the assembly, respectively. The proportion of repeat in RCC7840 (13.49%) is similar to the estimate (18%) obtained from short-read based prediction (Supplementary Table 1). The slight difference in estimates may be due to the binning and decontamination steps, which could have removed reads and affected repeats. There may also be a loss of repeat density caused by the assembly process, that could merge repeats artificially reducing the repeats content in the assembly. Additionally, the estimation methods differ, with the other one relying on k-mers extracted from short reads only.

Repetitive elements were softmasked in the genome to prevent them from interfering with gene predictions.

Structural annotation of the RCC7840 genome was performed using the funannotate pipeline in combination with BRAKER3. Transcript evidence was obtained from RNA-seq

data, which were first assembled using RNAspades and then integrated through *PASA* within the funannotate workflow. The *PASA* approach allows accurate modeling of gene structures by leveraging spliced transcript alignments, providing annotations consistent with experimentally observed transcripts. Gene models from *PASA*, *BRAKER3*, and protein alignments were combined using **EvidenceModeler (EVM)**, which allows assigning different weights to the various types of evidence. To reflect the higher reliability of transcript-supported predictions, greater weight was assigned to features derived from *PASA* and *BRAKER3*, while lower weight was given to predictions based solely on homology and ab initio (Supplementary Table 2). This ensures that gene models supported by transcript evidence are prioritized in the final annotation. Each prediction tool predicted a close number of gene models (11,000 to 12,000) except for Augustus and Augustus HiQ which predicted a much lower number of gene models (around 7,000 and 3,000 respectively). Evidence Modeler (EVM) combined these predictions into a set of 15,555 gene models (Supplementary Table 2). The final gene models were further refined by removing duplicates and adding tRNA genes, resulting in a total of **15,857** gene models. Functional annotation was performed using InterProScan, EggNOG-mapper, Phobius, SignalP6 and BarrNap. Functional annotations are available in the form of a *tsv* file, which can be then easily parsed. These results could be further validated by assessing transcriptome support. Mapping RNA-seq reads back to the predicted gene models or protein provides a direct measure of gene prediction quality, indicating the fraction of gene models supported by transcription. In addition, the proportion of genes with confident EggNOG assignment serves as a complementary validation. Together, these transcriptomic and functional validations strengthen confidence in the structural and functional annotation.

To further assess the quality of the annotation, I used BUSCO on the predicted proteome. The results indicated a higher level of completeness (C: 91.5%) compared to the genome assembly alone (C: 90.7%), with the same duplication rate (D: 2.1%) but a higher fragmented rate and lower missing rate (F: 6.1%, M: 2.4%). This suggests that the annotation process successfully identified additional conserved genes that were not fully captured in the assembly.

3.2 Assembly and Annotation Pipeline

Based on my benchmarking of RCC7840, I developed two separate pipelines: one for genome assembly and another for genome annotation. This separation was intended to enhance flexibility, ease of maintenance, and usability for both workflows. Additionally, having distinct pipelines allows users to manually adjust and curate the assembly before initiating annotation, and also enables the reannotation of existing genomes when needed. To ensure reproducibility and facilitate access for the broader scientific community, the pipelines used for genome assembly and annotation are publicly available. Researchers can access the genome assembly pipeline at <https://github.com/MaiDeme/genomeAssembly>, and the annotation pipeline at <https://github.com/MaiDeme/genomeAnnotation>.

These pipelines were developed using the workflow management system Snakemake v8.30.0. The choice of Snakemake was made because of its simple syntax and its resemblance to Python, making it easier for users familiar with Python programming. I also used singularity (apptainer version 1.4.0.1.el9) containers technology to ensure reproducibility and simplify the installation on different systems.

The first pipeline was designed with multiple possible paths (Figure 8), as no single tool or strategy is optimal for all data. Therefore several alternative tools were included to allow flexibility. Both pipelines allow users to choose their parameters and configuration with a config file. The assembly pipeline consists of four steps, (1) basecalling, read correction and binning, (2)

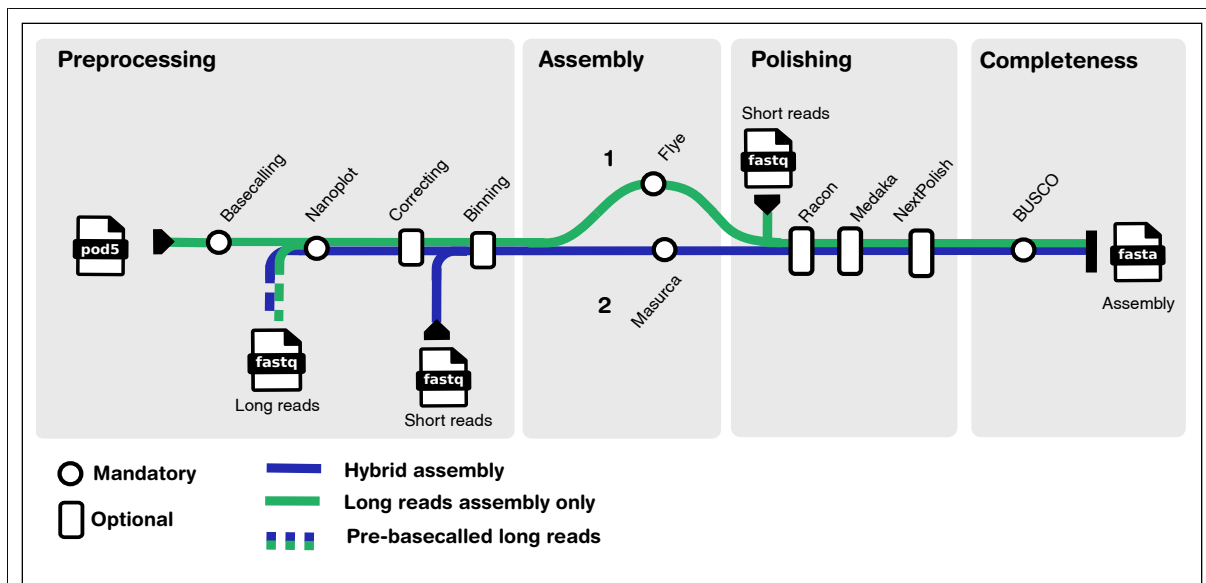


Figure 8: Overview of the assembly pipeline.

The correcting step cannot be done at the same time as choosing the MaSuRCA assembler. Correcting, binning and polishing are optional steps. The assembly can be done with either Flye (1) or MaSuRCA (2).

assembly using either Flye or MaSuRCA followed by (3) optional polishing, and (4) assembly quality assessment using BUSCO.

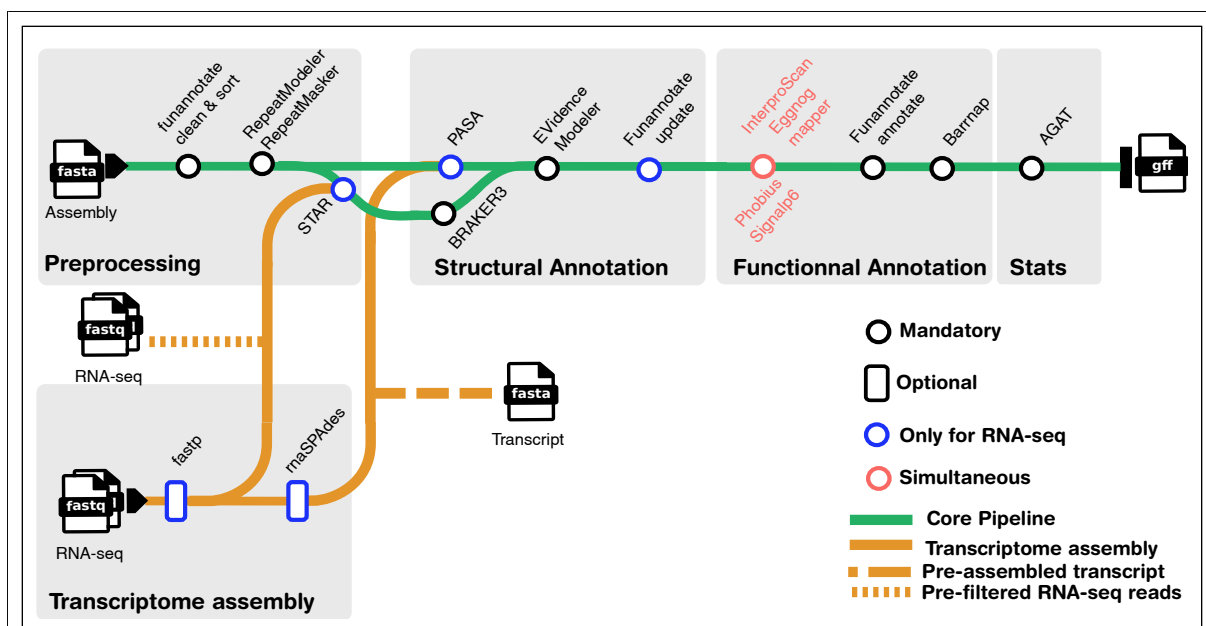


Figure 9: Overview of the annotation pipeline.

The pipeline can be run with or without RNA-seq data. The pipeline splits into two parallel branches: BRAKER3 is always run and PASA is additionally run only if RNA-seq data is available. Interproscan, EggNOG-mapper, Phobius and SignalP6 are parallelized and run on the same time. Barnap can be run at anytime after the preprocessing as it only need the assembly but if represented at the end for simplicity. PASA and EvidenceModeler are wrapped in funannotate.

The annotation pipeline (Figure 9) consists of five steps, (1) transcriptome assembly when RNA-seq data is available, (2) cleaning and repeat masking, (3) gene prediction using

BRAKER3 and funannotate (4) functional annotation using InterProScan, EggNOG-mapper, Phobius, SignalP6 and BarrNap, and (5) gene annotation statistics using AGAT.

If RNA-seq data is given, the pipeline will first assemble the transcriptome using RNAs-pades. The assembled transcripts are then used to predict gene models with PASA. You can also provide additional protein sequences to improve the gene prediction. These proteins will be added to the initial database (OrthoDB and Uniprot) used by BRAKER3 and funannotate. You can still use the pipeline without any RNA-seq data, in this case, the pipeline will skip the transcriptome assembly and PASA step and run BRAKER accordingly with only protein sequences.

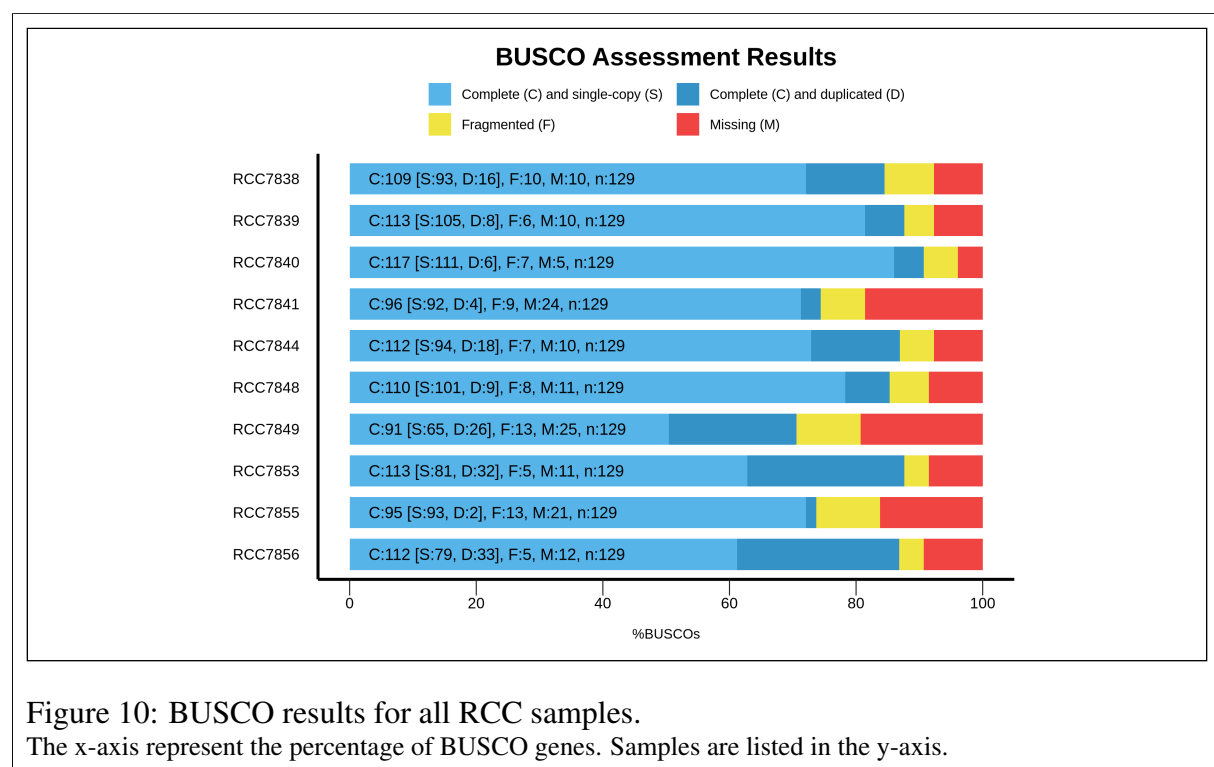
3.3 Application of the Assembly and Annotation pipelines to a broad range of algal species

The pipelines I developed was also tested by my supervisor as an alpha version. These tests demonstrated that the pipeline can be executed on the same machine by different users, without requiring previous knowledge of the Snakemake language, the underlying tools, or involvement in its implementation.

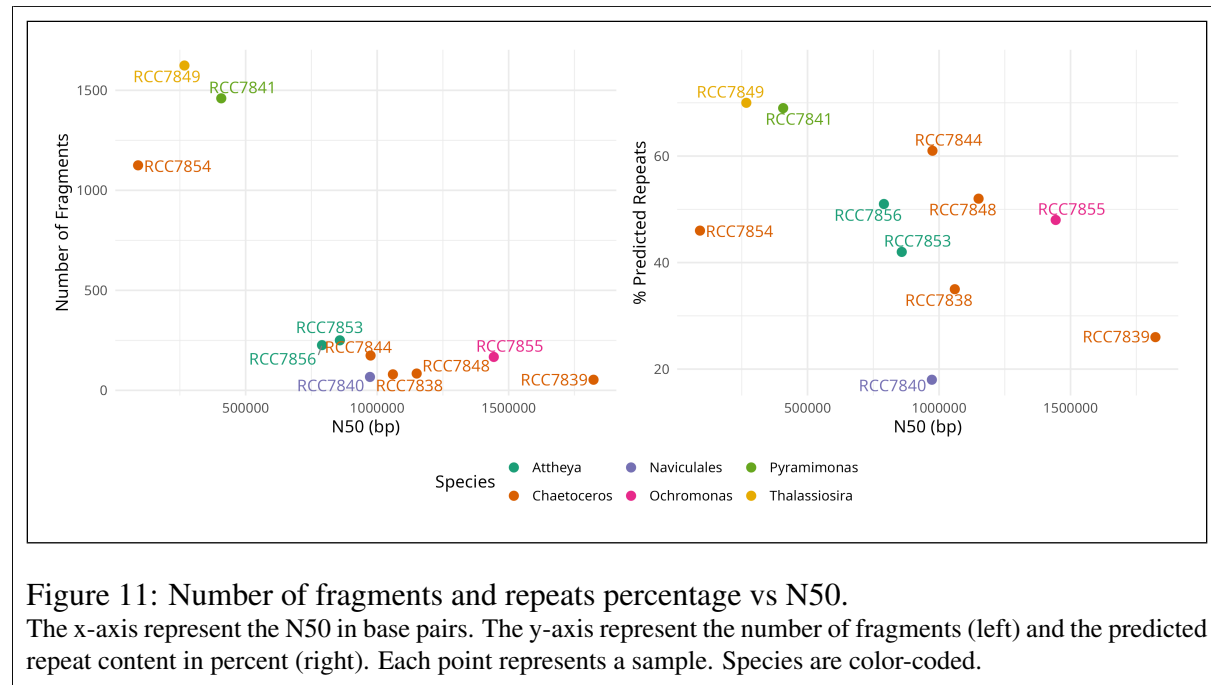
The analyses presented here were conducted using a previous version of the pipeline (v2 binning), since obtaining the most recent statistics is time-consuming. Accordingly, unless specified, the results reported below correspond to this earlier version.

I tested the pipelines with algal species belonging to different groups (i.e. diatoms, golden-brown algae, green algae) that were sequenced by the HEAL team (Supplementary Table 1).

All genomes were assembled successfully, with BUSCO completeness scores above 70%. Genome sizes varied considerably across samples, ranging from 30-80 Mbp in some species to 100-400 Mbp in others. Repeat content was highly variable, from 18% in RCC7840 to 77% in RCC7847 (Supplementary Table 1).



Most samples exhibited well-assembled genomes with good contiguity, though some samples (RCC7841, RCC7849, RCC7855) were more fragmented and had more than 15% missing BUSCO genes Figure 10, Figure 11. The poor assembly of RCC7854 (Supplementary Table 4), which was performed using fewer sequencing reads (Supplementary Figure 2), illustrates the impact of sequencing depth on assembly quality, resulting in a highly fragmented genome.



This can be explained by their high repeat content, RCC7849 and RCC7841 have the highest repeat content (70% and 77% respectively), which makes their assembly more difficult to resolve (Figure 11).

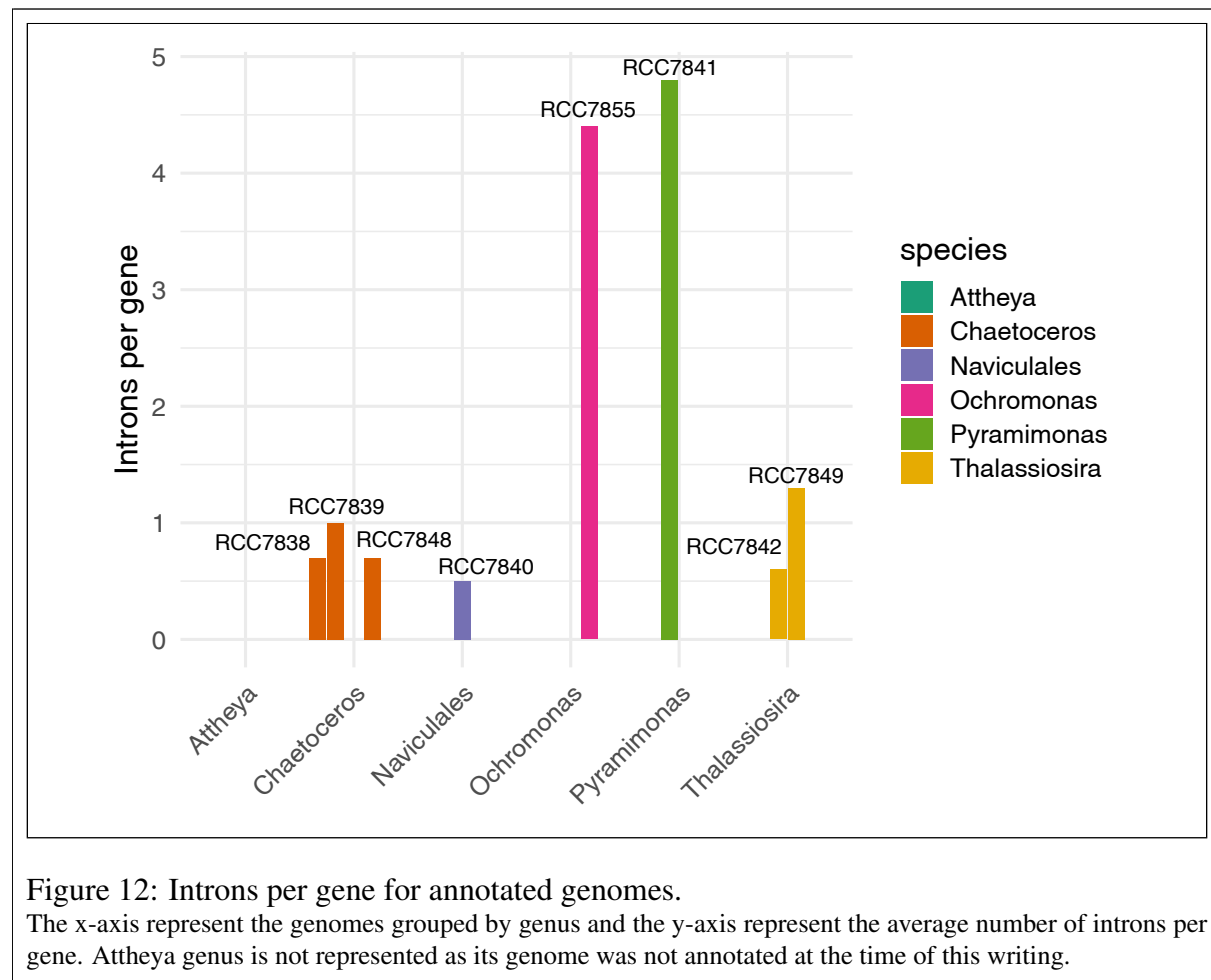
I tried to resolve this problem by augmenting the coverage of these samples, indeed Masurca is applying a cutoff on the coverage, retaining the longest reads. By default MaSuRCA recommends using 25x as more could lead to much longer computation and worse assembly. I tried to increase the coverage of RCC7838 to 25x, 30x and 35x, but it only worsened the assembly leading to more fragments and a slightly bigger assembly size (Table 7).

Coverage	N50	Size (Mb)	Fragments
25x	1 432 988	63.3	69
30x	1 302 569	64.2	75
35x	1 310 365	65.0	83

Table 7: Assembly statistics of RCC7838 with different Nanopore coverage. Assembly done with MaSuRCA (cabog) assembler on v2 reads.

For all samples tested, we had a huge part that was unclassified reinforcing our hypothesis of novel repeat families. These unclassified sequences could represent new, lineage-specific repeats that have not yet been characterized. Beyond annotation completeness, this is important because repeats are increasingly recognized as playing roles in genome regulation, chromatin organization, and gene expression [61, 62]. Therefore, exploring these unclassified elements could provide new insights into the diversity of repeated elements and how they affect the biology of these organisms.

Each species was found to have between 15,000 and 30,000 identified genes (Supplementary Table 3), which is consistent with the typical range reported for published annotated algal genomes [63]. The annotations are not limited to single-exon genes; exon-intron structures were also detected, indicating that the annotations capture meaningful gene models rather than being overly simplistic. We also compared structural features of the predicted gene models across the *Chaetoceros* genomes. The average number of introns per gene was highly similar among samples of the same genus (Figure 12, Supplementary Table 3), but different when compared to other species, reflecting both the conservation within the genus and divergence between species.



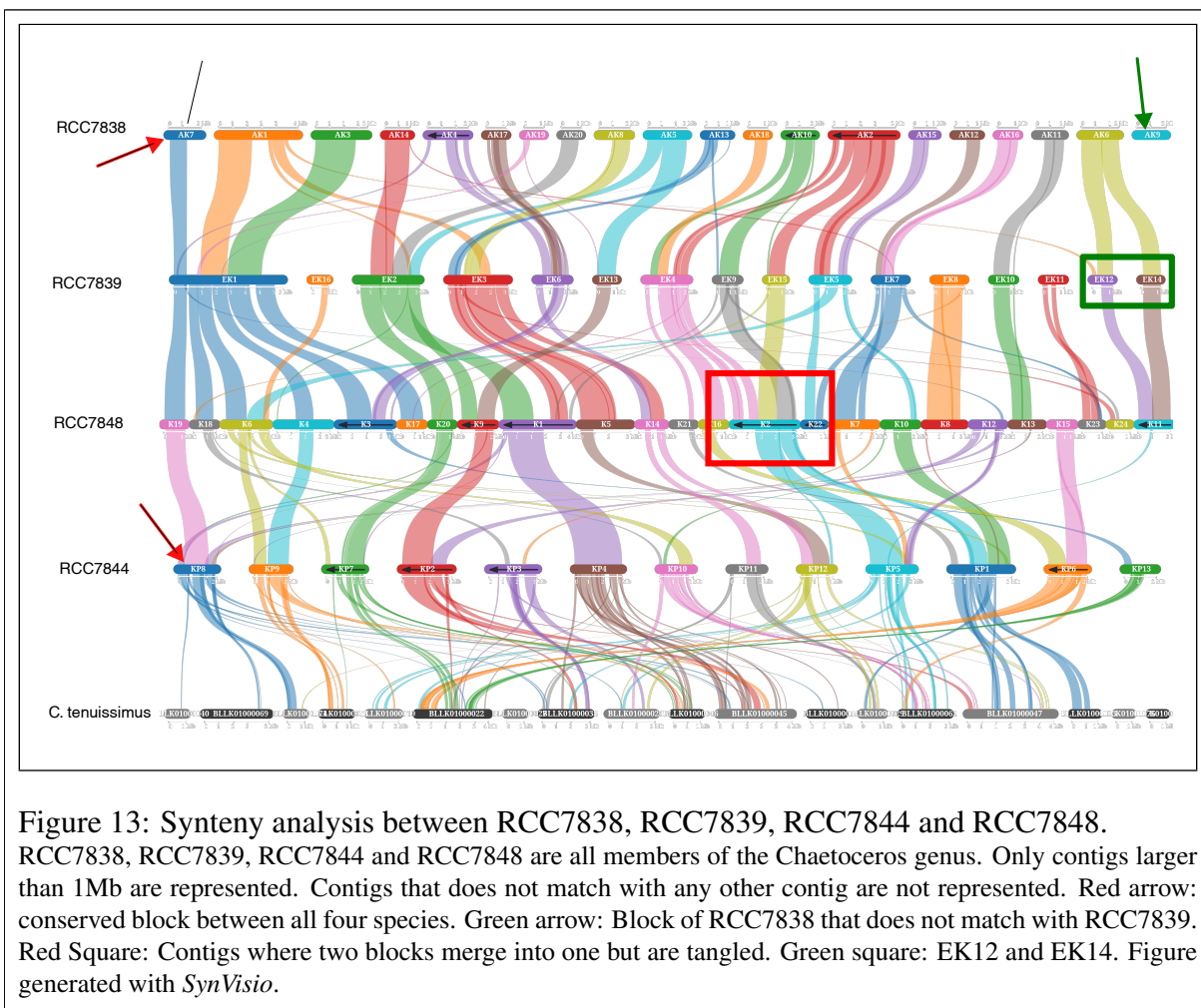
3.4 Synteny Analysis

To investigate the accuracy of the genome, I did a synteny analysis of four *Chaetoceros* genomes (RCC7838, RCC7839, RCC7844, RCC7848) using MCSanX, and compared them to the published *Chaetoceros tenuissimus* genome available at [GCA_021927905.1](https://www.ncbi.nlm.nih.gov/assembly/GCA_021927905.1) (Figure 13). We used newly generated assembly and annotation with the last version of the pipelines. High conservation of collinear blocks among the RCC strains suggests reliable assemblies and consistent annotations (e.g. red arrow on Figure 13).

Some blocks, such as EK12, EK14 (green square in Figure 13), may represent larger contigs, as they both match closely to a larger contigs, AK16, of another species. Certain blocks are conserved across multiple species but are located on large contigs with neighboring blocks that

do not match, potentially reflecting misassemblies or genomic rearrangements (red square in Figure 13).

Blocks of synteny are less conserved when compared to *C. tenuissimus* which may reflect both biological variability within the *Chaetoceros* genus, such as genomic rearrangements and lineage-specific events, as well as potential biases related to assembly quality. A phylogeny was done by a collaborator which showed that *C. tenuissimus* might be more closely related to RCC7838 and RCC7839 than RCC7844 and RCC7848. In addition, one block in RCC7838 (green arrow in Figure 13) does not match with RCC7839, but is present in the figure which means it does match but with other contig of another species. To further clarify these relationships, reordering the synteny figure to place *C. tenuissimus* in the middle could provide a more informative visualization of block conservation across all strains.



For the organelle genomes, Mauve enabled whole-genome alignments to assess synteny between members of the *Chaetoceros* genus and a more distantly related species, RCC7840. Figure 14 shows that between species of the same genus, the synteny is well conserved. RCC7839 and RCC7844 appear less well aligned but because the genome is circular, it is just because the starting point of the genome is different. However when comparing another genus, RCC7840, the order is not preserved at all which is expected as they are more distant.

Despite the pipeline being functional and producing reliable results, several technical limitations were identified. The analyses required considerable RAM and storage (100GB of RAM and more than 500GB of storage), largely because of reference databases and the buildup of

redundant intermediate files across multiple steps. Potential improvements could include relying on smaller or more targeted databases, exploring alternative decontamination tools that are less resource-intensive and systematically implementing cleaning scripts to remove large redundant files. In parallel, efforts are underway to containerize both pipelines. While the assembly pipeline was relatively straightforward to package, the annotation pipeline remains more challenging due to its multiple dependencies and the need to link external large databases. Completing this process will facilitate deployment across systems, improve reproducibility, and make the workflows more accessible to other users.

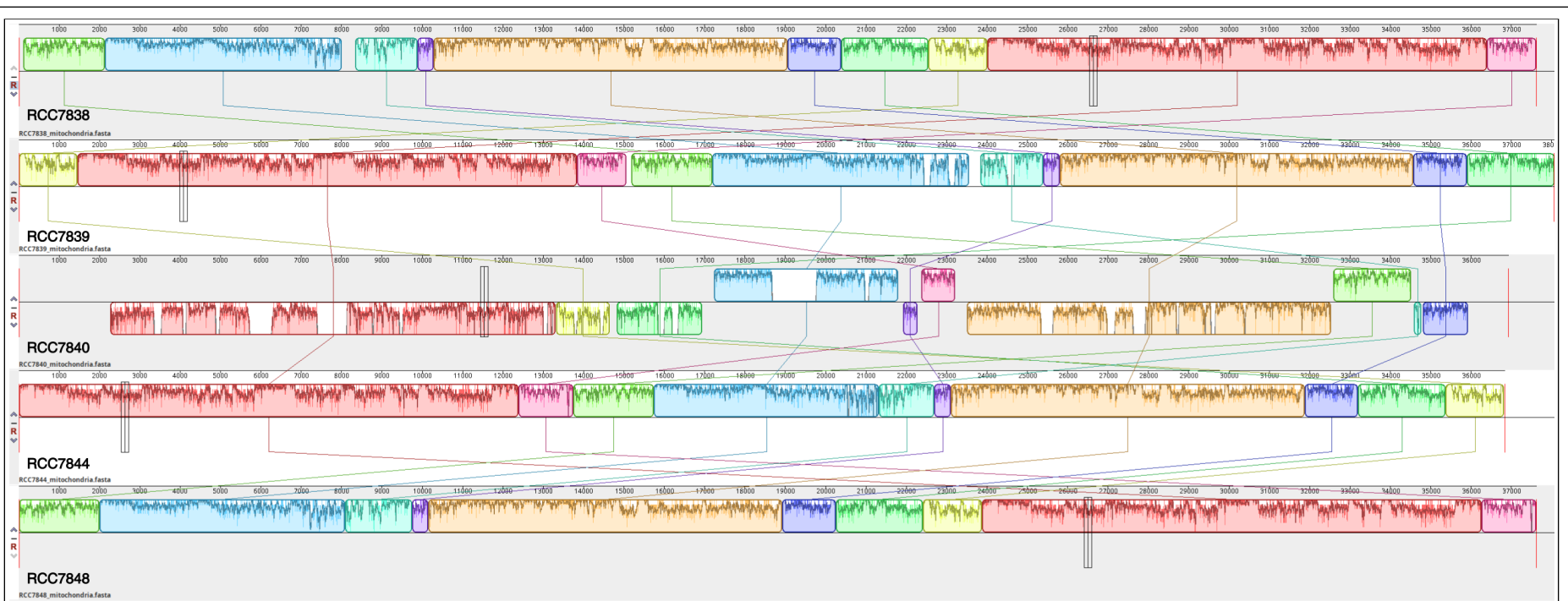


Figure 14: Mauve alignment of Chaetoceros Mitochondria Genome.

From top to bottom: RCC7838, RCC7839, RCC7840, RCC7844, RCC7848 All are member of the Chaetoceros genus except RCC7840. Colored block are syntenic blocks in the mitochondrial genome. Same color between two blocks means they are syntenic. Figure generated with *Mauve*.

4 Conclusion

The main purpose of this internship was to develop and implement pipelines for genome assembly and annotation of algal species. Application of these workflows to a diverse set of algal genomes demonstrated their ability to generate high-quality assemblies and annotations, capturing a significant number of genes, as well as identifying repeat elements including a notable fraction of unclassified sequences, potentially representing novel families. Although the pipelines are fully functional, technical limitations were noted, including high RAM usage and storage requirements.

Future improvements could focus on optimizing memory usage, providing containers and user-friendly installation, removing large intermediate files and testing the pipelines on additional algal species and datasets. Overall, these pipelines provide a robust and flexible framework that will facilitate genomic studies of algae and contribute to expanding the currently limited genomic resources for these ecologically and evolutionarily important organisms.

5 Internship Assessment and Personal Contributions

This internship allowed me to significantly strengthen my technical and methodological skills.

On the technical side, I gained hands-on experience with essential bioinformatics tools and programming languages. I worked with Python and R for data visualization, while using Snake-make to design reproducible workflows. I also improved my proficiency in the Unix environment, developing autonomy in managing files, running pipelines, and working on servers.

From a biological perspective, I deepened my understanding of genomics particularly in relation to algal genomes. I learned more about genome structure, and annotation processes. This experience allowed me to better connect computational analyses with biological questions. I gained significant hands-on experience in handling large genomic datasets, performing quality control, and interpreting results in a biological context.

In terms of interpersonal skills, working within a public laboratory gave me the opportunity to collaborate with researchers from different backgrounds that had interests and expertise in various domains. This experience enhanced my communication skills in english, as I had to explain my work and results to both computational and biological scientists.

Finally, this internship represents a decisive step in my professional development. It confirmed my motivation to stay in the public research sector for the moment. By working on a algal genomes, I not only developed strong computational skills but also gained a deeper appreciation for the biological complexity and diversity of these organisms. This experience has solidified my interest in pursuing a career in bioinformatics, particularly in the field of genomics.

References

- [1] Burkhard Büdel, Thomas Friedl, and Wolfram Beyschlag, eds. *Biology of Algae, Lichens and Bryophytes*. Berlin, Heidelberg: Springer, 2024. ISBN: 978-3-662-65711-9 978-3-662-65712-6. DOI: [10.1007/978-3-662-65712-6](https://doi.org/10.1007/978-3-662-65712-6).
- [2] J. David Allan, Maria M. Castillo, and Krista A. Capps. “Primary Producers”. In: *Stream Ecology : Structure and Function of Running Waters*. Ed. by J. David Allan, María M. Castillo, and Krista A. Capps. Cham: Springer International Publishing, 2021, pp. 141–176. ISBN: 978-3-030-61286-3. DOI: [10.1007/978-3-030-61286-3_6](https://doi.org/10.1007/978-3-030-61286-3_6).
- [3] Russell Leonard Chapman. “Algae: The World’s Most Important “Plants”—an Introduction”. In: *Mitigation and Adaptation Strategies for Global Change* 18.1 (Jan. 1, 2013), pp. 5–12. ISSN: 1573-1596. DOI: [10.1007/s11027-010-9255-9](https://doi.org/10.1007/s11027-010-9255-9).
- [4] Chaitanya Kumar et al. “Cultivation of Algae: Techniques and Challenges”. In: *Value Added Products From Bioalgae Based Biorefineries: Opportunities and Challenges*. Ed. by Shailendra Kumar Arya, Madhu Khatri, and Gursharan Singh. Singapore: Springer Nature, 2024, pp. 43–65. ISBN: 978-981-97-1662-3. DOI: [10.1007/978-981-97-1662-3_3](https://doi.org/10.1007/978-981-97-1662-3_3).
- [5] Erik R. Hanschen and Shawn R. Starkenburg. “The State of Algal Genome Quality and Diversity”. In: *Algal Research* 50 (Sept. 1, 2020), p. 101968. ISSN: 2211-9264. DOI: [10.1016/j.algal.2020.101968](https://doi.org/10.1016/j.algal.2020.101968).
- [6] Debashish Bhattacharya et al. “Why We Need More Algal Genomes”. In: *Journal of Phycology* 51.1 (2015), pp. 1–5. ISSN: 1529-8817. DOI: [10.1111/jpy.12267](https://doi.org/10.1111/jpy.12267).
- [7] *Horizontal Evolution of Algal Lifestyles (HEAL)*. LCQB. URL: <https://lcqb.fr/groupes/horizontal-evolution-of-algal-lifestyles-heal/>.
- [8] Jay Shendure et al. “DNA Sequencing at 40: Past, Present and Future”. In: *Nature* 550.7676 (Oct. 19, 2017), pp. 345–353. ISSN: 0028-0836, 1476-4687. DOI: [10.1038/nature24286](https://doi.org/10.1038/nature24286).
- [9] *GenBank and WGS Statistics*. URL: <https://www.ncbi.nlm.nih.gov/genbank/statistics/>.
- [10] *Sequencing Quality Scores*. URL: <https://www.illumina.com/science/technology/next-generation-sequencing/plan-experiments/quality-scores.html>.
- [11] *Nanopore Sequencing Accuracy*. Oxford Nanopore Technologies. URL: <https://nanoporetech.com/platform/accuracy>.
- [12] Heena Satam et al. “Next-Generation Sequencing Technology: Current Trends and Advancements”. In: *Biology* 12.7 (July 13, 2023), p. 997. ISSN: 2079-7737. DOI: [10.3390/biology12070997](https://doi.org/10.3390/biology12070997).
- [13] Todd J. Treangen and Steven L. Salzberg. “Repetitive DNA and Next-Generation Sequencing: Computational Challenges and Solutions”. In: *Nature Reviews Genetics* 13.1 (Jan. 2012), pp. 36–46. ISSN: 1471-0064. DOI: [10.1038/nrg3117](https://doi.org/10.1038/nrg3117).

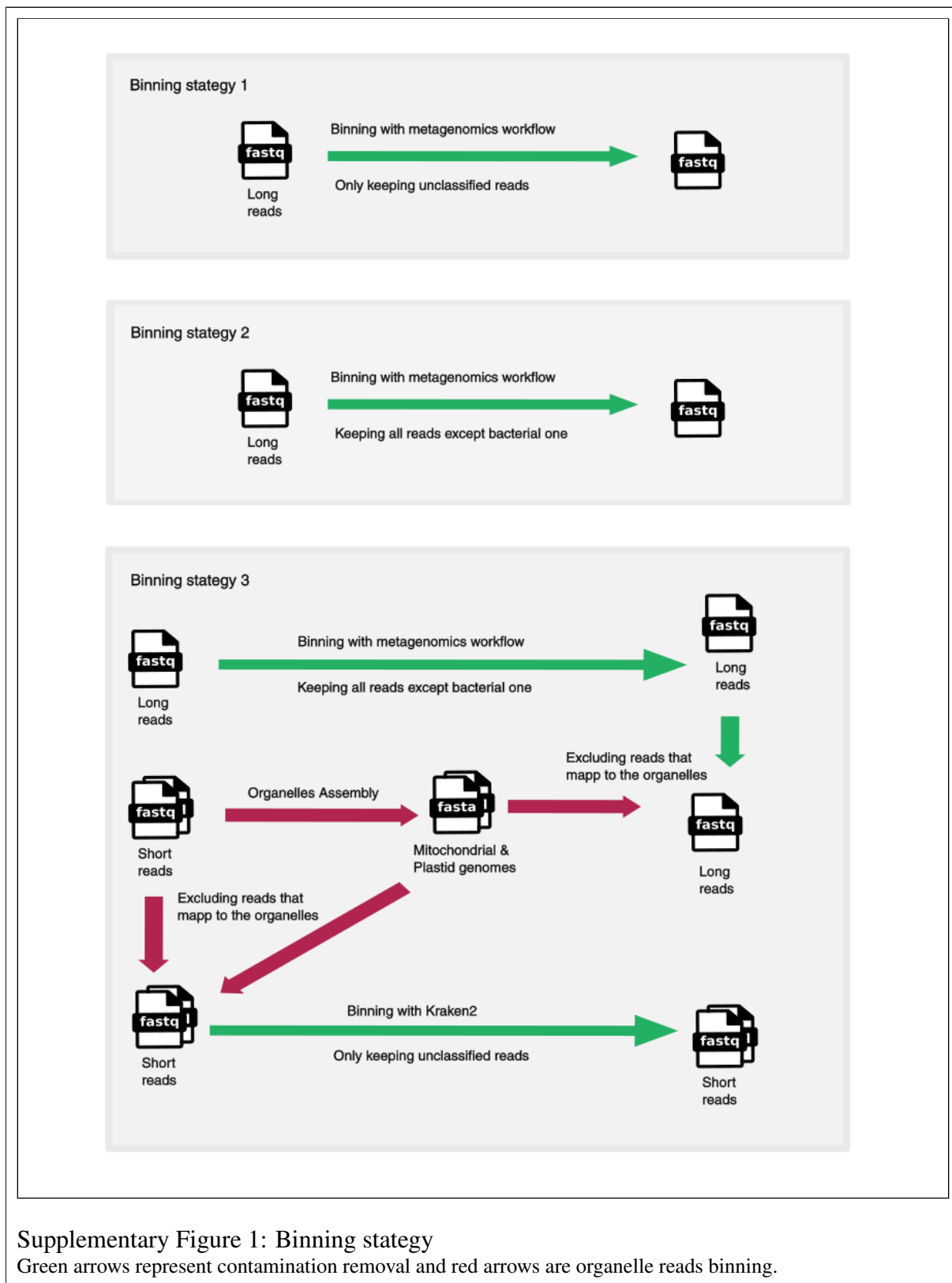
- [14] Hyungtaek Jung et al. “Twelve Quick Steps for Genome Assembly and Annotation in the Classroom”. In: *PLOS Computational Biology* 16.11 (Nov. 12, 2020), e1008325. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1008325](https://doi.org/10.1371/journal.pcbi.1008325).
- [15] Ole K Tørresen et al. “Tandem Repeats Lead to Sequence Assembly Errors and Impose Multi-Level Challenges for Genome and Protein Databases”. In: *Nucleic Acids Research* 47.21 (Dec. 2, 2019), pp. 10994–11006. ISSN: 0305-1048. DOI: [10.1093/nar/gkz841](https://doi.org/10.1093/nar/gkz841).
- [16] Heng Li and Richard Durbin. “Genome Assembly in the Telomere-to-Telomere Era”. In: *Nature Reviews Genetics* 25.9 (Sept. 2024), pp. 658–670. ISSN: 1471-0064. DOI: [10.1038/s41576-024-00718-w](https://doi.org/10.1038/s41576-024-00718-w).
- [17] Frédéric Chaux et al. “Extraction and Selection of High-Molecular-Weight DNA for Long-Read Sequencing from *Chlamydomonas Reinhardtii*”. In: *PLOS ONE* 19.2 (Feb. 8, 2024), e0297014. ISSN: 1932-6203. DOI: [10.1371/journal.pone.0297014](https://doi.org/10.1371/journal.pone.0297014).
- [18] T. Rhyker Ranallo-Benavidez, Kamil S. Jaron, and Michael C. Schatz. “GenomeScope 2.0 and Smudgeplot for Reference-Free Profiling of Polyploid Genomes”. In: *Nature Communications* 11.1 (Mar. 18, 2020), p. 1432. ISSN: 2041-1723. DOI: [10.1038/s41467-020-14998-3](https://doi.org/10.1038/s41467-020-14998-3).
- [19] *Dorado Documentation*. URL: <https://dorado-docs.readthedocs.io/en/latest/>.
- [20] Dominik Stanojević et al. *Telomere-to-Telomere Phased Genome Assembly Using Error-Corrected Simplex Nanopore Reads*. May 21, 2024. DOI: [10.1101/2024.05.18.594796](https://doi.org/10.1101/2024.05.18.594796). Pre-published.
- [21] Yuansheng Liu et al. “Repeat and Haplotype Aware Error Correction in Nanopore Sequencing Reads with DeChat”. In: *Communications Biology* 7.1 (Dec. 19, 2024), pp. 1–10. ISSN: 2399-3642. DOI: [10.1038/s42003-024-07376-y](https://doi.org/10.1038/s42003-024-07376-y).
- [22] *Epi2me-Labs/Wf-Metagenomics*. EPI2ME, May 9, 2025. URL: <https://github.com/epi2me-labs/wf-metagenomics>.
- [23] Nicolas Dierckxsens, Patrick Mardulyn, and Guillaume Smits. “NOVOPlasty: De Novo Assembly of Organelle Genomes from Whole Genome Data”. In: *Nucleic Acids Research* 45.4 (Feb. 28, 2017), e18. ISSN: 0305-1048. DOI: [10.1093/nar/gkw955](https://doi.org/10.1093/nar/gkw955).
- [24] Mikhail Kolmogorov et al. “Assembly of Long, Error-Prone Reads Using Repeat Graphs”. In: *Nature Biotechnology* 37.5 (May 2019), pp. 540–546. ISSN: 1546-1696. DOI: [10.1038/s41587-019-0072-8](https://doi.org/10.1038/s41587-019-0072-8).
- [25] Sergey Koren et al. “Canu: Scalable and Accurate Long-Read Assembly via Adaptive k-Mer Weighting and Repeat Separation”. In: *Genome Research* 27.5 (May 2017), pp. 722–736. ISSN: 1088-9051. DOI: [10.1101/gr.215087.116](https://doi.org/10.1101/gr.215087.116).
- [26] Aleksey V. Zimin et al. “Hybrid Assembly of the Large and Highly Repetitive Genome of *Aegilops Tauschii*, a Progenitor of Bread Wheat, with the MaSuRCA Mega-Reads Algorithm”. In: *Genome Research* 27.5 (May 2017), pp. 787–792. ISSN: 1549-5469. DOI: [10.1101/gr.213405.116](https://doi.org/10.1101/gr.213405.116).
- [27] Jiang Hu et al. “NextDenovo: An Efficient Error Correction and Accurate Assembly Tool for Noisy Long Reads”. In: *Genome Biology* 25.1 (Apr. 26, 2024), p. 107. ISSN: 1474-760X. DOI: [10.1186/s13059-024-03252-4](https://doi.org/10.1186/s13059-024-03252-4).

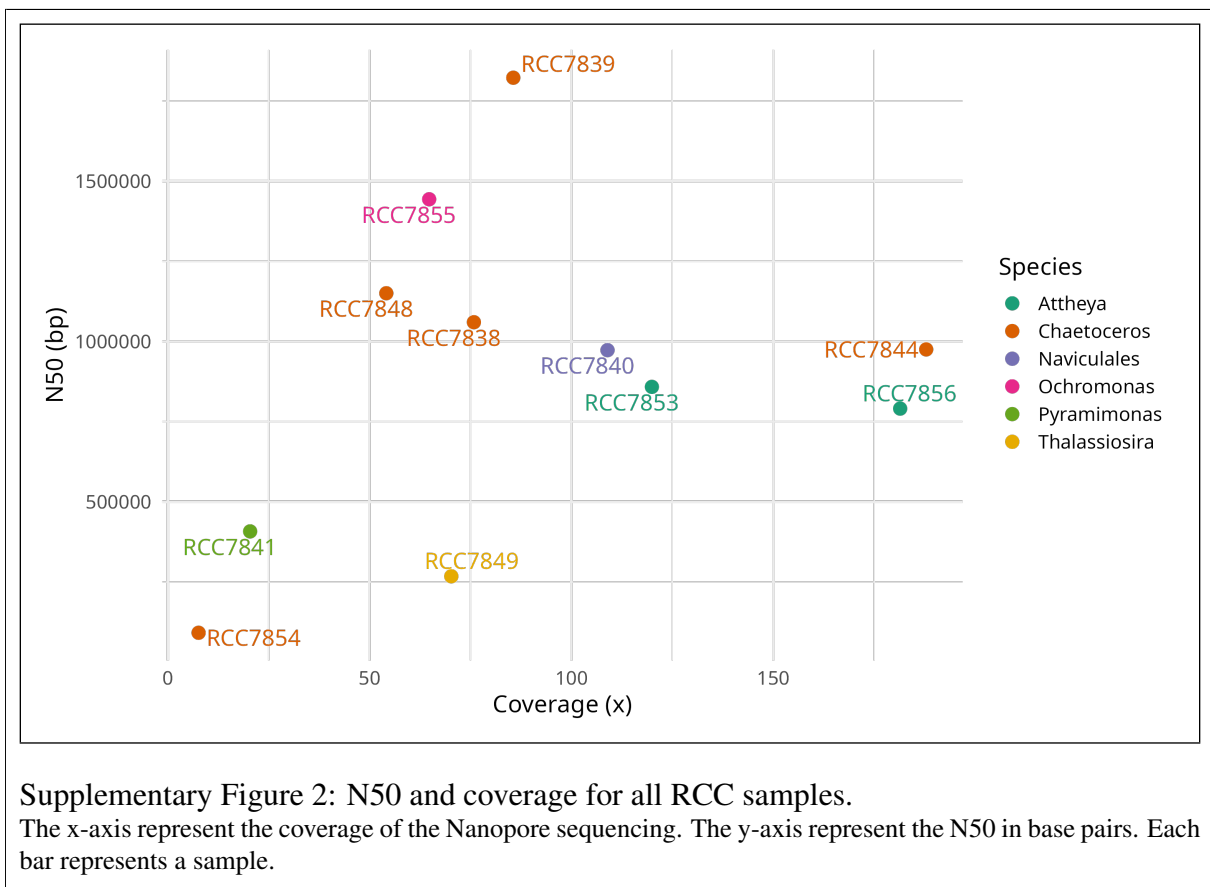
- [28] Haoyu Cheng et al. “Haplotype-Resolved de Novo Assembly Using Phased Assembly Graphs with Hifiasm”. In: *Nature Methods* 18.2 (Feb. 2021), pp. 170–175. ISSN: 1548-7105. DOI: [10.1038/s41592-020-01056-5](https://doi.org/10.1038/s41592-020-01056-5).
- [29] Robert Vaser and Mile Šikić. “Time- and Memory-Efficient Genome Assembly with Raven”. In: *Nature Computational Science* 1.5 (May 2021), pp. 332–336. ISSN: 2662-8457. DOI: [10.1038/s43588-021-00073-4](https://doi.org/10.1038/s43588-021-00073-4).
- [30] Tu Luan et al. “Benchmarking Short and Long Read Polishing Tools for Nanopore Assemblies: Achieving near-Perfect Genomes for Outbreak Isolates”. In: *BMC Genomics* 25.1 (July 8, 2024), p. 679. ISSN: 1471-2164. DOI: [10.1186/s12864-024-10582-x](https://doi.org/10.1186/s12864-024-10582-x).
- [31] Robert Vaser et al. “Fast and Accurate de Novo Genome Assembly from Long Uncorrected Reads”. In: *Genome Research* 27.5 (May 2017), pp. 737–746. ISSN: 1088-9051. DOI: [10.1101/gr.214270.116](https://doi.org/10.1101/gr.214270.116).
- [32] Nanoporetech/Medaka. Oxford Nanopore Technologies, Aug. 15, 2025. URL: <https://github.com/nanoporetech/medaka>.
- [33] Jiang Hu et al. “NextPolish: A Fast and Efficient Genome Polishing Tool for Long-Read Assembly”. In: *Bioinformatics* 36.7 (Apr. 1, 2020), pp. 2253–2255. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/btz891](https://doi.org/10.1093/bioinformatics/btz891).
- [34] Felipe A. Simão et al. “BUSCO: Assessing Genome Assembly and Annotation Completeness with Single-Copy Orthologs”. In: *Bioinformatics* 31.19 (Oct. 1, 2015), pp. 3210–3212. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/btv351](https://doi.org/10.1093/bioinformatics/btv351).
- [35] Shifu Chen et al. “Fastp: An Ultra-Fast All-in-One FASTQ Preprocessor”. In: *Bioinformatics* 34.17 (Sept. 1, 2018), pp. i884–i890. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/bty560](https://doi.org/10.1093/bioinformatics/bty560).
- [36] Elena Bushmanova et al. “rnaSPAdes: A de Novo Transcriptome Assembler and Its Application to RNA-Seq Data”. In: *GigaScience* 8.9 (Sept. 1, 2019), giz100. ISSN: 2047-217X. DOI: [10.1093/gigascience/giz100](https://doi.org/10.1093/gigascience/giz100).
- [37] Alexander Dobin et al. “STAR: Ultrafast Universal RNA-seq Aligner”. In: *Bioinformatics (Oxford, England)* 29.1 (Jan. 1, 2013), pp. 15–21. ISSN: 1367-4811. DOI: [10.1093/bioinformatics/bts635](https://doi.org/10.1093/bioinformatics/bts635).
- [38] Jullien M. Flynn et al. *RepeatModeler2: Automated Genomic Discovery of Transposable Element Families*. Nov. 26, 2019. DOI: [10.1101/856591](https://doi.org/10.1101/856591). Pre-published.
- [39] Robert Hubley Arian Smit. *RepeatMasker: RepeatMasker Is a Program That Screens DNA Sequences for Interspersed Repeats and Low Complexity DNA Sequences*. Version 4.1.8. Mar. 12, 2025. URL: <http://www.repeatmasker.org>.
- [40] Lars Gabriel et al. “BRAKER3: Fully Automated Genome Annotation Using RNA-seq and Protein Evidence with GeneMark-ETP, AUGUSTUS and TSEBRA”. In: *bioRxiv: The Preprint Server for Biology* (Feb. 29, 2024), p. 2023.06.10.544449. ISSN: 2692-8205. DOI: [10.1101/2023.06.10.544449](https://doi.org/10.1101/2023.06.10.544449).
- [41] Tomas Bruna, Alexandre Lomsadze, and Mark Borodovsky. “A New Gene Finding Tool GeneMark-ETP Significantly Improves the Accuracy of Automatic Annotation of Large Eukaryotic Genomes”. In: *bioRxiv: The Preprint Server for Biology* (Apr. 17, 2024), p. 2023.01.13.524024. ISSN: 2692-8205. DOI: [10.1101/2023.01.13.524024](https://doi.org/10.1101/2023.01.13.524024).

- [42] Mario Stanke et al. “AUGUSTUS: A Web Server for Gene Finding in Eukaryotes”. In: *Nucleic Acids Research* 32 (Web Server issue July 1, 2004), W309–W312. ISSN: 0305-1048. DOI: [10.1093/nar/gkh379](https://doi.org/10.1093/nar/gkh379).
- [43] W. H. Majoros, M. Pertea, and S. L. Salzberg. “TigrScan and GlimmerHMM: Two Open Source *Ab Initio* Eukaryotic Gene-Finders”. In: *Bioinformatics* 20.16 (Nov. 1, 2004), pp. 2878–2879. ISSN: 1367-4811, 1367-4803. DOI: [10.1093/bioinformatics/bth315](https://doi.org/10.1093/bioinformatics/bth315).
- [44] Ian Korf. “Gene Finding in Novel Genomes”. In: *BMC Bioinformatics* 5.1 (May 14, 2004), p. 59. ISSN: 1471-2105. DOI: [10.1186/1471-2105-5-59](https://doi.org/10.1186/1471-2105-5-59).
- [45] Brian J. Haas et al. “Improving the Arabidopsis Genome Annotation Using Maximal Transcript Alignment Assemblies”. In: *Nucleic Acids Research* 31.19 (Oct. 1, 2003), pp. 5654–5666. ISSN: 1362-4962. DOI: [10.1093/nar/gkg770](https://doi.org/10.1093/nar/gkg770).
- [46] Jonathan M. Palmer and Jason E. Stajich. *Funannotate*. Version 1.8.16. Aug. 2023. URL: <https://github.com/nextgenusfs/funannotate>.
- [47] Philip Jones et al. “InterProScan 5: Genome-Scale Protein Function Classification”. In: *Bioinformatics* 30.9 (May 1, 2014), pp. 1236–1240. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/btu031](https://doi.org/10.1093/bioinformatics/btu031).
- [48] Carlos P Cantalapiedra et al. “eggNOG-mapper v2: Functional Annotation, Orthology Assignments, and Domain Prediction at the Metagenomic Scale”. In: *Molecular Biology and Evolution* 38.12 (Dec. 1, 2021), pp. 5825–5829. ISSN: 1537-1719. DOI: [10.1093/molbev/msab293](https://doi.org/10.1093/molbev/msab293).
- [49] Lukas Käll, Anders Krogh, and Erik L. L Sonnhammer. “A Combined Transmembrane Topology and Signal Peptide Prediction Method”. In: *Journal of Molecular Biology* 338.5 (May 14, 2004), pp. 1027–1036. ISSN: 0022-2836. DOI: [10.1016/j.jmb.2004.03.016](https://doi.org/10.1016/j.jmb.2004.03.016).
- [50] Felix Teufel et al. “SignalP 6.0 Predicts All Five Types of Signal Peptides Using Protein Language Models”. In: *Nature Biotechnology* 40.7 (July 2022), pp. 1023–1025. ISSN: 1546-1696. DOI: [10.1038/s41587-021-01156-3](https://doi.org/10.1038/s41587-021-01156-3).
- [51] Torsten Seemann. *Tseemann/Barrnap*. June 19, 2025. URL: <https://github.com/tseemann/barrnap>.
- [52] *Welcome to AGAT’s Documentation! - AGAT*. URL: <https://nbisweden.github.io/AGAT/>.
- [53] Martin Steinegger and Johannes Söding. “MMseqs2 Enables Sensitive Protein Sequence Searching for the Analysis of Massive Data Sets”. In: *Nature Biotechnology* 35.11 (Nov. 2017), pp. 1026–1028. ISSN: 1546-1696. DOI: [10.1038/nbt.3988](https://doi.org/10.1038/nbt.3988).
- [54] Yupeng Wang. *Wyp1125/MCScanX*. Aug. 19, 2025. URL: <https://github.com/wyp1125/MCScanX>.
- [55] *SynVisio : Synteny Browser*. URL: <https://synvisio.github.io/#/>.
- [56] Aaron C.E. Darling et al. “Mauve: Multiple Alignment of Conserved Genomic Sequence With Rearrangements”. In: *Genome Research* 14.7 (July 2004), pp. 1394–1403. ISSN: 1088-9051. DOI: [10.1101/gr.2289704](https://doi.org/10.1101/gr.2289704).
- [57] Nora Zidane et al. “Accurate Genotyping of Three Major Respiratory Bacterial Pathogens with ONT R10.4.1 Long-Read Sequencing”. In: *Genome Research* 35.8 (Aug. 1, 2025), pp. 1758–1766. ISSN: 1549-5469. DOI: [10.1101/gr.279829.124](https://doi.org/10.1101/gr.279829.124).

- [58] Luc Cornet and Denis Baurain. “Contamination Detection in Genomic Data: More Is Not Enough”. In: *Genome Biology* 23.1 (Feb. 21, 2022), p. 60. ISSN: 1474-760X. DOI: [10.1186/s13059-022-02619-9](https://doi.org/10.1186/s13059-022-02619-9).
- [59] Monique Turmel, Christian Otis, and Claude Lemieux. “Divergent Copies of the Large Inverted Repeat in the Chloroplast Genomes of Ulvophyceae Green Algae”. In: *Scientific Reports* 7.1 (Apr. 20, 2017), p. 994. ISSN: 2045-2322. DOI: [10.1038/s41598-017-01144-1](https://doi.org/10.1038/s41598-017-01144-1).
- [60] Richard Kolodner and K. K. Tewari. “Inverted Repeats in Chloroplast DNA from Higher Plants*”. In: *Proceedings of the National Academy of Sciences of the United States of America* 76.1 (Jan. 1979), pp. 41–45. ISSN: 0027-8424. DOI: [10.1073/pnas.76.1.41](https://doi.org/10.1073/pnas.76.1.41).
- [61] Xingyu Liao et al. “Repetitive DNA Sequence Detection and Its Role in the Human Genome”. In: *Communications Biology* 6.1 (Sept. 19, 2023), p. 954. ISSN: 2399-3642. DOI: [10.1038/s42003-023-05322-y](https://doi.org/10.1038/s42003-023-05322-y).
- [62] Valentina Casa and Davide Gabellini. “A Repetitive Elements Perspective in Polycomb Epigenetics”. In: *Frontiers in Genetics* 3 (Oct. 8, 2012), p. 199. ISSN: 1664-8021. DOI: [10.3389/fgene.2012.00199](https://doi.org/10.3389/fgene.2012.00199).
- [63] Natalia Nenasheva et al. “Annotation of Protein-Coding Genes in 49 Diatom Genomes from the Bacillariophyta Clade”. In: *Scientific Data* 12.1 (June 11, 2025), p. 985. ISSN: 2052-4463. DOI: [10.1038/s41597-025-05306-z](https://doi.org/10.1038/s41597-025-05306-z).
- [64] *Bioinformatics Web Server - University of Greifswald*. URL: https://bioinf.uni-greifswald.de/bioinf/partitioned_odb12/.

A Supplementary Figures





B Supplementary Tables

Sample	Species	Genome Size (Mbp)	Repeats (%)	Reads (Mbp)	N50 (kbp)	Process Reads (Mbp)	N50 (kpb)
RCC7838	Chaetoceros_sp	54-59	35	0.892	17	0.851	16
RCC7839	Chaetoceros_sp	46-47	26	0.837	11	0.668	11
RCC7840	Naviculales_XX_sp	38-39	18	0.653	12	0.588	12
RCC7841	Pyramimonas_sp	409-464	69	2.7	26	1.880	23
RCC7844	Chaetoceros_sp	105-114	61	5.3	13	4.223	9
RCC7847	Pyramimonas_sp	399	77	0.226	17	0.214	17
RCC7848	Chaetoceros_sp	78-81	52	0.963	15	0.880	15
RCC7849	Thalassiosira_sp	284-303	70	7.6	9	7.598	9
RCC7853	Attheya_sp	92-97	46	1.2	17	1.073	16
RCC7854	Chaetoceros_sp	107-110	67	3.6	14	0.838	6.3
RCC7855	Ochromonas_sp	109-119	48	2.5	7	2.420	7
RCC7856	Attheya_sp	103-107	51	2.7	17	2.042	16

Supplementary Table 1: Sequencing Summary of the RCC samples.

Features	Augustus	HiQ	GeneMark	GlimmerHMM	BRAKER3	PASA	SNAP	EVM
Weights	1	2	1	1	10	10	1	-
Genes	7,514	3,825	15,445	13,507	15,485	11,596	14,002	15,555
Mean CDS length	753.012	707.884	924.548	773.576	1000.64	944.196	680.476	973.119

Supplementary Table 2: Number of genes predicted by each tool in the annotation pipeline for RCC7840.

EVM = Evidence Modeler

HiQ = Augustus High Quality

Final column correspond to the final number of genes after the consensus step with Evidence Modeler.

Sample	Genes	intron/gene	single-exon genes	gene length	exon length	intron length	exons coverage %	genes coverage %
RCC7838	21 748	0.7	12 877	1 712	965	137	54.2	56.7
RCC7839	15 506	1	8 495	1 580	854	140	51.5	55
RCC7840	15 857	0.5	10 445	1 760	1 120	149	67.5	70
RCC7841	28 764	4.8	3 509	5 806	261	883	10.6	39.8
RCC7844	27 751	0.6	17 077	1 631	954	133	44.2	46
RCC7848	21 133	0.7	12 226	1 715	951	139	55.3	58.6
RCC7849	31 429	1.3	14 047	1766	712	128	18.4	19.9
RCC7855	20 828	4.4	6 500	3 276	309	223	25.1	47.6

Supplementary Table 3: Statistics of the annotations of the RCC samples.

The columns *intron/gene*, *single-exon genes*, *gene length*, *exon length*, *intron length*, *exons coverage %* and *genes coverage %* represent the average number of introns per gene, the number of single-exon genes, the average gene length, the average exon length, the average intron length, the percentage of exons covered by the annotation and the percentage of genes covered by the annotation respectively.

Sample	Size	Contigs	N50	contigs >1Mb	C %	D %	F %	M %
RCC7838	65 612 755	65	1 621 431	22	84,5	12,4	7,8	7,8
RCC7839	44 499 341	65	1 434 746	14	87,6	6,2	4,7	7,8
RCC7840	39 834 854	82	927 216	11	90,7	4,7	5,4	3,9
RCC7841	419 825 127	1 460	407 344	42	74,4	3,1	7	18,6
RCC7844	98 425 205	174	975 036	24	86,80	14	5,4	7,8
RCC7848	62 214 495	84	1 150 305	17	85,3	7	6,2	8,5
RCC7849	331 868 888	1 624	267 272	9	70,5	20,2	10,1	19,4
RCC7853	119 821 622	250	858 222	30	87,6	24,8	3,9	8,5
RCC7854	57 046 502	1 125	91 071	0				
RCC7855	143 260 363	167	1 443 507	42	73,6	1,6	10,1	16,3
RCC7856	114 471 606	226	790 395	27	86,8	25,6	3,9	9,3

Supplementary Table 4: Statistics of the assemblies of the RCC samples.

The columns C, D, F and M represent the percentage of complete, fragmented, duplicated and missing BUSCO genes respectively.

C Commands

C.1 Assembly

C.1.1 Dorado

```
dorado basecaller sup --device cuda:0 --emit-fastq input.folder.pod5 > output
```

C.1.2 Herro

```
dorado correct reads.fastq > corrected.fasta -device cuda:0
```

C.1.3 Dechat

```
dechat -i reads.fastq -o corrected_reads -t <threads>
```

C.1.4 Binning

Taxonomic classification was performed using the Nextflow pipeline

```
nextflow run epi2me-labs/wf-metagenomics --fastq input.fasqt -profile singularity
--classifier kraken2 --include_reads_assignments True --out-dir out -r
v2.13.0 --sample name -database_set <db>
```

C.1.4.1 Binning v1

The first strategy, (binning v1) was done using:

```
grep -v ^"U" out/reads_assignments/name_lineages.kraken2.assignments.tsv |
awk '{{print $2}}' > output.ids
seqtk subseq raw_reads output.ids > reads.fastq
```

C.1.4.2 Binning v2

To filter reads (binning v2), we used the command:

```
grep -v "Bacteria" out/reads_assignments/name_lineages.kraken2.assignments.tsv|
  awk '{{print $2}}' > output.ids
seqtk subseq raw_reads output.ids > reads.fastq
```

C.1.4.3 Binning v3

For the v3 strategy, we used this command to filter shorts reads:

```
kraken2 --db db --threads <threads> --output name.kraken --report name.report
  --paired --gzip-compressed --unclassified-out reads_kraken#.fq
  reads_1.fq.gz reads_2.fq.gz
```

Mitochondrial assembly

Config file:

```
# Novoplastry Mitochondria configuration file
Project:
-----
Project name      = mitochondria
Type              = mito
Genome Range      = 20000-49000
K-mer             = 33
Max memory        =
Extended log      = 0
Save assembled reads = no
Seed Input        = mito.seed
Extend seed directly = no
Reference sequence =
Variance detection =
Chloroplast sequence =

Dataset 1:
-----
Read Length       = 151
Insert size       = 300
Platform          = illumina
Single/Paired     = PE
Combined reads    =
Forward reads     = "forward_reads.fq.gz"
Reverse reads     = "reverse_reads.fq.gz"
Store Hash        = no

Heteroplasmy:
-----
MAF               =
HP exclude list   =
```

```

PCR-free          =

Optional:
-----
Insert size auto   = yes
Use Quality Scores = no
Reduce ambiguous N's =
Output path        = "novoplasty/"

```

Chloroplast assembly

Config file:

```

# Novoplasty Chloroplast configuration file
Project:
-----
Project name       = chloroplast
Type               = chloro
Genome Range       = 120000-200000
K-mer              = 33
Max memory         =
Extended log       = 0
Save assembled reads = no
Seed Input         = chloro.seed
Extend seed directly = no
Reference sequence =
Variance detection =
Chloroplast sequence =

Dataset 1:
-----
Read Length        = 151
Insert size        = 300
Platform           = illumina
Single/Paired      = PE
Combined reads     =
Forward reads      = "forward_reads.fq.gz"
Reverse reads      = "reverse_reads.fq.gz"
Store Hash         = no

Heteroplasmy:
-----
MAF                =
HP exclude list    =
PCR-free           =

Optional:
-----
Insert size auto   = yes

```

```

Use Quality Scores      = no
Reduce ambiguous N's    =
Output path             = choloroplast/

```

Command: perl NOVOPlasty4.3.5.pl -c config.file

The seed used for the mitochondrial assembly was the cox1 sequence from the <https://www.ncbi.nlm.nih.gov/nucore/LC537470.1?from=25295&to=26785> genome and for the chloroplastid assembly was the PsaA sequence from the genome https://www.ncbi.nlm.nih.gov/nucore/NC_045244.1.

C.1.5 Flye

Flye was runned using two different command:

```
flye --nano-hq long_reads.fastq.gz --out-dir flye_output --threads <threads>
--scaffold
```

for raw reads and

```
flye --nano-corr corrected_reads.fastq.gz --out-dir flye_output --threads <threads>
--scaffold
```

for corrected reads.

C.1.6 Canu

```
canu -p sample -d output genomeSize=39m -nanopore long_reads.fastq.gz
```

C.1.7 Masurca

Masurca was runned using a config file like the following, modifying the FLYE_ASSEMBLY option to 1 for final Flye assembly and to 0 for CABOG assembly:

```

DATA
PE = pe 500 50 reads_1.fq.gz reads_2.fq.gz
NANOPORE = long_reads.fastq.gz
END
PARAMETERS
USE_GRID = 0
LHE_COVERAGE = 25
CA_PARAMETERS = cgwErrorRate=0.15
CLOSE_GAPS = 1
NUM_THREADS = 32
JF_SIZE = 200000000
SOAP_ASSEMBLY = 0
FLYE_ASSEMBLY = 0
END

```

Command: masurca config.txt
./assemble.sh.

C.1.8 NextDenovo

Config file:

```
[General]
job_type = sge # here we use SGE to manage jobs
job_prefix = nextDenovo
task = all
rewrite = yes
deltmp = yes
parallel_jobs = 22
input_type = raw
read_type = ont # clr, ont, hifi
input_fofn = input.fofn
workdir = RCC7840

[correct_option]
read_cutoff = 1k
genome_size = 39m # estimated genome size
sort_options = -m 50g -t 30
minimap2_options_raw = -t 8
pa_correction = 5
correction_options = -p 30

[assemble_option]
minimap2_options_cns = -t 8
nextgraph_options = -a 1
```

Command:

```
ls <long_reads.fastq.gz> > input.fofn
nextDenovo run.cfg
```

C.1.9 hifiasm

```
hifiasm -o assembly --ont -t <threads> <long_reads.fastq.gz>
```

C.1.10 Raven

```
raven --threads <threads> <long_reads.fastq.gz> > assembly.fasta
```

C.1.11 BUSCO

```
busco -i <assembly.fasta> -l eukaryota_odb12 -m genome -tar -c <threads> -o BUSCO
```

C.1.12 Correction Assembly

Racon was run using the following command four times.

```
minimap2 -ax map-ont <assembly.fasta> <long_reads.fastq.gz> assembly.sorted.bam
racon <long_reads.fastq.gz> <assembly.sorted.bam> <assembly.fasta> > corrected.fasta
```

Then, medaka was run using the command

```
medaka_consensus -i <reads.fastq.gz> -d <assembly.fasta> -o medaka_output
-t <threads>
```

Lastly, we finished the polishing with nextpolish using the config file:

```
[General]
job_type = local
job_prefix = nextPolish
task = best
rewrite = yes
deltmp = yes
rerun = 3
parallel_jobs = 6
multithread_jobs = 5
genome = ./raw.genome.fasta
genome_size = auto
workdir = ./01_rundir
polish_options = -p {multithread_jobs}

[sgs_option]
sgs_fofn = ./sgs.fofn
sgs_options = -max_depth 100 -bwa
```

and run with the command

```
ls <*.fastq.gz> > sgs.fofn
nextPolish run.cfg
```

C.2 Annotation

C.2.1 Fastp and rnaspades

```
fastp -i <reads_1.fq.gz> -I <reads_2.fq.gz> -o <reads_1.filtered.fq.gz>
-O <reads_2.filtered.fq.gz> --detect_adapter_for_pe
rnaspades.py -1 <rna_r.fastq.gz> -2 <rna_f.fastq.gz> -o <rnaspades_output>
-t <threads>
```

C.2.2 STAR

```
STAR --runThreadN <threads> --genomeDir --runMode genomeGenerate --genomeDir
output/index --outFileNamePrefix output/ --genomeFastaFiles genome.fasta
STAR --runThreadN <threads> --genomeDir output/index --readFilesIn rna_r.fastq.gz
rna_f.fastq.gz --readFilesCommand zcat --outSAMtype BAM Unsorted
--outSAMstrandField intronMotif --outFileNamePrefix output/
```

C.2.3 Renaming and sorting

```
funannotate clean -i <assembly.fasta> -o <output.fasta> --exhaustive
funannotate sort -i <output.fasta> -o <sorted.fasta> --minlen 1
```


C.2.4 Repeat

```
RepeatModeler/BuildDatabase -name <db_name> <assembly.fasta>
RepeatModeler/RepeatModeler -database <db_name> -threads <threads>
RepeatMasker -pa <threads> -lib <consensi.fa.classfied> --xsmall --no_is --gff
```

C.2.5 Braker3

Braker was run when no RNA-seq was available using:

```
braker.pl --genome <assembly.fasta> --workingdir=<output_dir> --species
<species_name> --gff3 --threads <threads> --prot_seq <protein_sequences.fasta>
--useexisting
```

When RNA-seq was available, we used the command:

```
braker.pl --genome <assembly.fasta> --workingdir=<output_dir> --species
<species_name> --gff3 --threads <threads> --prot_seq <protein_sequences.fasta>
--bam <star.bam> --useexisting
```

The protein sequences were the odb12 database [64] reduce to eukaryota and if available, additional protein concatenated to the fasta file.

C.2.6 Funannotate (Structural Annotation)

The whole structural annotation encapsulated in the Funnannotate pipeline was run using the command (if RNA-seq was available):

```
funannotate train -i <assembly.fasta> -o <output_dir> --species
<species_name> --trinity <transcriptome.fasta> --cpus 20 -l <rna_left.fq.gz>
-r <rna_right.fq.gz>
funannotate predict -i <assembly.fasta> --rna_bam <star.bam>
--protein_evidence <protein_evidence1> <protein_evidence2> --pasa_gff <pasa.gff>
--transcript_evidence <transcriptome.fasta> -o <output_dir> -s
<species_name> --cpus 20 --other_gff <braker.gff> --organism other --ploidy 2
--busco_db eukaryota --name <species.name> -w augustus:1 hiq:2 genemark:1
pasa:10 snap:1 glimmerhmm:1 proteins:5 transcripts:1 other_pred1:2
funannotate update -i <output_dir> --cpus 20
```

If RNA-seq was not available, the command was:

```
funannotate predict -i <assembly.fasta> --protein_evidence
<protein_evidence1> <protein_evidence2> -o <output_dir> -s <species_name>
--cpus 20 --other_gff <braker.gff> --organism other --ploidy 2 --busco_db
eukaryota --name <species.name> -w augustus:1 hiq:2 genemark:1 snap:1
glimmerhmm:1 proteins:5 other_pred1:2
```

C.2.7 Functional Annotation

C.2.7.1 Eggno-mapper

```
emapper.py -m diamon -i <protein.fasta> --out_dir <output_dir> --cpu 20 --override
```

C.2.7.2 InterProScan

```
interproscan.sh -i <protein.fasta> -d <output_dir> -f XML -goterms -pa
```

C.2.7.3 SignalP6

```
signalp6 -fasta <protein.fasta> --output_dir <output_dir> -org euk -format txt
```

C.2.7.4 Phobius

```
phobius.pl <protein.fasta> -short > <output_dir>/phobius.out
```

C.2.7.5 Funnannotate

Funnannotate was run to integrate functional annotation using:

```
funannotatate annotate -i <fun_dir> --iprscan <interpscan.xml> --eggnog  
  <eggnog.emapper.annotations> --signalp <predictions_results.txt> --phobius  
  <phobius.out> --cpus 20 --busco_db eukaryota
```

C.2.8 rRNA prediction

Barrnap (rRNA prediction) was run using the command:

```
barrnap --kingdom euk ---threads 20 <assembly.fasta> > <output_dir>/barrnap.gff
```

C.2.9 Annotation statistics

```
agat_sp_statistics.pl -gff <gff_file> -g <genome.fasta> -d -p -o <output_dir>
```